Contents

1 Algebra **2**
- 1.1 Binary Exponent . . . 2
- 1.2 Chinese_reminder_theorem . . . 2
- 1.3 Chinese_reminder_theorem . . . 2
- 1.4 Diophantine . . . 2
- 1.5 Diophantine . . . 2
- 1.6 Euler's_totient_function . . . 2
- 1.7 Euler's_totient_function_1to_n . . . 2
- 1.8 Extended_Euclid . . . 2
- 1.9 Factorial mod P . . . 2
- 1.10 Fibonacci Number Faster . . . 2
- 1.11 Lucas Theorem . . . 3
- 1.12 Modular_inverse . . . 3
- 1.13 Number of Divisors . . . 3
- 1.14 Sieve upto 1e9 . . . 3
- 1.15 Sum of Diviors . . . 4

2 DP **4**
- 2.1 Coin 1 . . . 4
- 2.2 Coin 2 . . . 4
- 2.3 DC_dp . . . 4
- 2.4 Knapsack 01 . . . 4
- 2.5 Knapsack 02 . . . 5
- 2.6 Knuth's Optomization . . . 5
- 2.7 LCS . . . 5
- 2.8 Minimum sum Diff . . . 5
- 2.9 Subset sum . . . 6

3 Data Structure **6**
- 3.1 BIT 2D with Range Update and Range Query . . . 6
- 3.2 BIT with Range Update and Range Query . . . 6
- 3.3 BIT . . . 7
- 3.4 Dsu . . . 7
- 3.5 LCA . . . 7
- 3.6 MOs Algorithm . . . 7
- 3.7 MOs on tree . . . 8
- 3.8 MOs with Update . . . 8
- 3.9 Ordered Set . . . 9
- 3.10 Segment Tree Lazy . . . 9
- 3.11 Segment Tree . . . 9
- 3.12 Sparse table . . . 9
- 3.13 Trie . . . 9
- 3.14 segment tree 2(sum) . . . 10

4 Geometry **10**
- 4.1 2D point . . . 10
- 4.2 3D point . . . 10
- 4.3 Area of polygon . . . 10
- 4.4 Check Intersection . . . 10
- 4.5 Circle line intersection . . . 11
- 4.6 Common tengents of circle . . . 11
- 4.7 Convex hall . . . 11
- 4.8 Intersection of segments . . . 11
- 4.9 Line intersection . . . 12
- 4.10 Oriented area of a triangle . . . 12
- 4.11 check if belongs to convex poly . . . 12
- 4.12 common type . . . 12

5 Graph **13**
- 5.1 Articulation Points . . . 13
- 5.2 Bellman Ford . . . 13
- 5.3 Directed MST . . . 13
- 5.4 Floyd Warshall . . . 14
- 5.5 Johnson's Algorithm . . . 14
- 5.6 LCA . . . 15
- 5.7 Prim's MST . . . 15
- 5.8 SCC . . . 15
- 5.9 SPFA . . . 16

6 Linear Algebra **16**
- 6.1 Determinant . . . 16
- 6.2 Guass-jordan . . . 16

7 Matrix **16**
- 7.1 Integration (Simpsons) . . . 16
- 7.2 Inverse of A Matrix . . . 17
- 7.3 Matrix Exponentiation . . . 17

8 Notes **18**
- 8.1 Counting . . . 18
  - 8.1.1 Fibonacci . . . 18
- 8.2 Geometry . . . 18
  - 8.2.1 Triangle . . . 18
- 8.3 Series . . . 18
- 8.4 Sum . . . 18

9 QOL **18**
- 9.1 build . . . 18
- 9.2 debug . . . 18
- 9.3 stress . . . 18

10 String **19**
- 10.1 KMP . . . 19
- 10.2 Manachers . . . 19
- 10.3 String Hashing 2D . . . 19
- 10.4 String Hashing . . . 20
- 10.5 Suffix Array . . . 20

1 Algebra

1.1 Binary Exponent

```
/*Returns a^b in O(log(b))*/
ll binpowmod(ll a, ll b, ll m)
{
    ll res = 1;
    while (b > 0)
    {
        if (b & 1)
        {
            res *= a;
            res = res % m;
        }
        a *= a;
        a = a%m;
        b >>= 1;
    }
    return res;
}
```

1.2 Chinese_reminder_theorem

```
struct Congruence {
    long long a, m;
};
long long CRT(vector<Congruence> const&
 ↪  congruences) {
    long long M = 1;
    for (auto const& congruence : congruences) {
        M *= congruence.m;
    }

    long long solution = 0;
    for (auto const& congruence : congruences) {
        long long a_i = congruence.a;
        long long M_i = M / congruence.m;
        long long N_i = mod_inv(M_i, congruence.m);
        solution = (solution + a_i * M_i % M * N_i)
         ↪  % M;
    }
    return solution;
}
```

1.3 Chinese_reminder_theorem

Let $m = m_1 \cdot m_2 \cdots m_k$, where m_i are pairwise coprime. In addition to m_i, we are also given a system of congruences

$$\begin{cases} a \equiv a_1 \pmod{m_1} \\ a \equiv a_2 \pmod{m_2} \\ \quad\vdots \\ a \equiv a_k \pmod{m_k} \end{cases}$$

where a_i are some given constants. The original form of CRT then states that the given system of congruences always has *one and exactly one* solution modulo m.

1.4 Diophantine

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
bool find_any_solution(int a, int b, int c, int
 ↪  &x0, int &y0, int &g) {
    g = gcd(abs(a), abs(b), x0, y0);
    if (c % g) {
        return false;
    }
    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}
```

1.5 Diophantine

A Linear Diophantine Equation (in two variables) is an equation of the general form:

$$ax + by = c$$

where a, b, c are given integers, and x, y are unknown integers. Code gives a single solution for x and y for any other solution we can use:

$$x = x_0 + k \cdot \frac{b}{g}, \qquad y = y_0 - k \cdot \frac{a}{g}$$

1.6 Euler's_totient_function

```
/*Gives phi(n) in O(sqrt(n))*/
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}
```

1.7 Euler's_totient_function_1to_n

```
/*O(nloglog(n))*/
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

1.8 Extended_Euclid

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}
```

1.9 Factorial mod P

```
/*O(log_p (n))*/
int factmod(int n, int p) {
    vector<int> f(p);
    f[0] = 1;
    for (int i = 1; i < p; i++)
        f[i] = f[i-1] * i % p;

    int res = 1;
    while (n > 1) {
        if ((n/p) % 2)
            res = p - res;
        res = res * f[n%p] % p;
        n /= p;
    }
    return res;
}
```

1.10 Fibonacci Number Faster

```
#include<bits/stdc++.h>
using namespace std;

int fib(long long n, int mod) {
  assert (n >= 0);
  if (n <= 1) return n;
  int a = 0, b = 1;
  long long i = 1ll << (63 - __builtin_clzll(n) -
   ↪  1);
  for (; i; i >>= 1) {
    int na = (a *(long long) a + b *(long long) b)
     ↪  % mod;
    int nb = (2ll * a + b) * b % mod;
    a = na; b = nb;
    if (n & i) {
      int c = a + b; if (c >= mod) c -= mod;
      a = b; b = c;
    }
  }
  return b;
}
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  cout << fib(10, 100) << '\n';
  return 0;
}
```

1.11 Lucas Theorem

```
#include<bits/stdc++.h>
using namespace std;

const int N = 1e6 + 3, mod = 1e6 + 3;
using ll = long long;

template <const int32_t MOD>
struct modint {
  int32_t value;
  modint() = default;
  modint(int32_t value_) : value(value_) {}
  inline modint<MOD> operator + (modint<MOD> other) const { int32_t c = this->value + other.value; return modint<MOD>(c >= MOD ? c - MOD : c); }
  inline modint<MOD> operator - (modint<MOD> other) const { int32_t c = this->value - other.value; return modint<MOD>(c <    0 ? c + MOD : c); }
  inline modint<MOD> operator * (modint<MOD> other) const { int32_t c = (int64_t)this->value * other.value % MOD; return modint<MOD>(c < 0 ? c + MOD : c); }
  inline modint<MOD> & operator += (modint<MOD> other) { this->value += other.value; if (this->value >= MOD) this->value -= MOD; return *this; }
  inline modint<MOD> & operator -= (modint<MOD> other) { this->value -= other.value; if (this->value <    0) this->value += MOD; return *this; }
  inline modint<MOD> & operator *= (modint<MOD> other) { this->value = (int64_t)this->value * other.value % MOD; if (this->value < 0) this->value += MOD; return *this; }
  inline modint<MOD> operator - () const { return modint<MOD>(this->value ? MOD - this->value : 0); }
  modint<MOD> pow(uint64_t k) const { modint<MOD> x = *this, y = 1; for (; k; k >>= 1) { if (k & 1) y *= x; x *= x; } return y; }
  modint<MOD> inv() const { return pow(MOD - 2); } // MOD must be a prime
  inline modint<MOD> operator /  (modint<MOD> other) const { return *this *  other.inv(); }
  inline modint<MOD> operator /= (modint<MOD> other)       { return *this *= other.inv(); }
  inline bool operator == (modint<MOD> other) const { return value == other.value; }
  inline bool operator != (modint<MOD> other) const { return value != other.value; }
  inline bool operator < (modint<MOD> other) const { return value < other.value; }
  inline bool operator > (modint<MOD> other) const { return value > other.value; }
};
template <int32_t MOD> modint<MOD> operator * (int32_t value, modint<MOD> n) { return modint<MOD>(value) * n; }
template <int32_t MOD> modint<MOD> operator * (int64_t value, modint<MOD> n) { return modint<MOD>(value % MOD) * n; }
template <int32_t MOD> istream & operator >> (istream & in, modint<MOD> &n) { return in >> n.value; }
template <int32_t MOD> ostream & operator << (ostream & out, modint<MOD> n) { return out << n.value; }

using mint = modint<mod>;

struct combi{
  int n; vector<mint> facts, finvs, invs;
  combi(int _n): n(_n), facts(_n), finvs(_n), invs(_n){
    facts[0] = finvs[0] = 1;
    invs[1] = 1;
    for (int i = 2; i < n; i++) invs[i] =  invs[mod % i] * (-mod / i);
    for(int i = 1; i < n; i++){
      facts[i] = facts[i - 1] * i;
      finvs[i] = finvs[i - 1] * invs[i];
    }
  }
  inline mint fact(int n) { return facts[n]; }
  inline mint finv(int n) { return finvs[n]; }
  inline mint inv(int n) { return invs[n]; }
  inline mint ncr(int n, int k) { return n < k or k < 0 ? 0 : facts[n] * finvs[k] * finvs[n-k]; }
};
combi C(N);

// returns nCr modulo mod where mod is a prime
// Complexity: log(n)
mint lucas(ll n, ll r) {
  if (r > n) return 0;
  if (n < mod) return C.ncr(n, r);
  return lucas(n / mod, r / mod) * lucas(n % mod, r % mod);
}

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  cout << lucas(100000000, 2322) << '\n';
  return 0;
}
```

1.12 Modular_inverse

```
int x, y;
int g = extended_euclidean(a, m, x, y);
if (g != 1) {
    cout << "No solution!";
}
else {
    x = (x % m + m) % m;
    cout << x << endl;
}
```

1.13 Number of Divisors

```
long long numberOfDivisors(long long num) {
    long long total = 1;
    for (int i = 2; (long long)i * i <= num; i++) {
        if (num % i == 0) {
            int e = 0;
            do {
                e++;
                num /= i;
            } while (num % i == 0);
            total *= e + 1;
        }
    }
    if (num > 1) {
        total *= 2;
    }
    return total;
}
```

1.14 Sieve upto 1e9

```
#include<bits/stdc++.h>
using namespace std;

// credit: min_25
// takes 0.5s for n = 1e9
vector<int> sieve(const int N, const int Q = 17, const int L = 1 << 15) {
  static const int rs[] = {1, 7, 11, 13, 17, 19, 23, 29};
  struct P {
    P(int p) : p(p) {}
    int p; int pos[8];
  };
  auto approx_prime_count = [] (const int N) -> int {
    return N > 60184 ? N / (log(N) - 1.1)
                     : max(1., N / (log(N) - 1.11)) + 1;
  };

  const int v = sqrt(N), vv = sqrt(v);
  vector<bool> isp(v + 1, true);
  for (int i = 2; i <= vv; ++i) if (isp[i]) {
    for (int j = i * i; j <= v; j += i) isp[j] = false;
  }

  const int rsize = approx_prime_count(N + 30);
  vector<int> primes = {2, 3, 5}; int psize = 3;
  primes.resize(rsize);

  vector<P> sprimes; size_t pbeg = 0;
  int prod = 1;
  for (int p = 7; p <= v; ++p) {
    if (!isp[p]) continue;
    if (p <= Q) prod *= p, ++pbeg, primes[psize++] = p;
    auto pp = P(p);
    for (int t = 0; t < 8; ++t) {
      int j = (p <= Q) ? p : p * p;
      while (j % 30 != rs[t]) j += p << 1;
      pp.pos[t] = j / 30;
    }
    sprimes.push_back(pp);
  }

  vector<unsigned char> pre(prod, 0xFF);
```

```
  for (size_t pi = 0; pi < pbeg; ++pi) {
    auto pp = sprimes[pi]; const int p = pp.p;
    for (int t = 0; t < 8; ++t) {
      const unsigned char m = ~(1 << t);
      for (int i = pp.pos[t]; i < prod; i += p)
       ↪  pre[i] &= m;
    }
  }

  const int block_size = (L + prod - 1) / prod *
  ↪  prod;
  vector<unsigned char> block(block_size); unsigned
  ↪  char* pblock = block.data();
  const int M = (N + 29) / 30;

  for (int beg = 0; beg < M; beg += block_size,
  ↪  pblock -= block_size) {
    int end = min(M, beg + block_size);
    for (int i = beg; i < end; i += prod) {
      copy(pre.begin(), pre.end(), pblock + i);
    }
    if (beg == 0) pblock[0] &= 0xFE;
    for (size_t pi = pbeg; pi < sprimes.size();
    ↪  ++pi) {
      auto& pp = sprimes[pi];
      const int p = pp.p;
      for (int t = 0; t < 8; ++t) {
        int i = pp.pos[t]; const unsigned char m =
        ↪  ~(1 << t);
        for (; i < end; i += p) pblock[i] &= m;
        pp.pos[t] = i;
      }
    }
    for (int i = beg; i < end; ++i) {
      for (int m = pblock[i]; m > 0; m &= m - 1) {
        primes[psize++] = i * 30 +
        ↪  rs[__builtin_ctz(m)];
      }
    }
  }
  assert(psize <= rsize);
  while (psize > 0 && primes[psize - 1] > N)
  ↪  --psize;
  primes.resize(psize);
  return primes;
}

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int n, a, b; cin >> n >> a >> b;
  auto primes = sieve(n);
  vector<int> ans;
  for (int i = b; i < primes.size() && primes[i] <=
  ↪  n; i += a) ans.push_back(primes[i]);
  cout << primes.size() << ' ' << ans.size() <<
  ↪  '\n';
  for (auto x: ans) cout << x << ' '; cout << '\n';
  return 0;
}
// https://judge.yosupo.jp/problem/enumerate_primes
```

1.15 Sum of Diviors

```
long long SumOfDivisors(long long num) {
    long long total = 1;
    for (int i = 2; (long long)i * i <= num; i++) {
        if (num % i == 0) {
            int e = 0;
            do {
                e++;
                num /= i;
            } while (num % i == 0);
            long long sum = 0, pow = 1;
            do {
                sum += pow;
                pow *= i;
            } while (e-- > 0);
            total *= sum;
        }
    }
    if (num > 1) {
        total *= (1 + num);
    }
    return total;
}
```

2 DP

2.1 Coin 1

```
/*GReturn the fewest number of coins that you need
    to make up that amount. If that amount of money
 ↪  cannot be made up by any combination of the
 ↪  coins, return -1.*/
int coinChange(vector<int> &coins, int amount)
{
    if (amount < 1)
        return 0;
    vector<int> minCoinsDP(amount + 1, INT_MAX);
    minCoinsDP[0] = 0;
    for (int i = 1; i <= amount; i++)
    {
        for (int coin : coins)
        {
            if (coin <= i && minCoinsDP[i - coin]
            ↪  != INT_MAX)
            {
                minCoinsDP[i] = min(minCoinsDP[i],
                ↪  1 + minCoinsDP[i - coin]);
            }
        }
    }
    if (minCoinsDP[amount] == INT_MAX)
    {
        return -1;
    }
    return minCoinsDP[amount];
}
```

2.2 Coin 2

```
/*Return the number of combinations that make up
    that amount. If that amount of money cannot be
 ↪  made up by any combination of the coins, return
 ↪  0.*/
int change(int amount, vector<int> &coins)
{
    vector<vector<int>> dp(coins.size() + 1,
    ↪  vector<int>(amount + 1, 0));
    dp[0][0] = 1; // 1 comb: no coins for no amount
    for (int j = 1; j <= coins.size(); j++)
    {
        for (int i = 0; i <= amount; i++)
        {
            dp[j][i] = dp[j - 1][i]; // Exclude the
            ↪  current coin
            if (i >= coins[j - 1])
            {
                dp[j][i] += dp[j][i - coins[j -
                ↪  1]]; // Include the current coin
            }
        }
    }
    return dp[coins.size()][amount];
}
```

2.3 DC_dp

```
int m, n;
vector<long long> dp_before(n), dp_cur(n);
long long C(int i, int j);
void compute(int l, int r, int optl, int optr) {
    if (l > r)
        return;
    int mid = (l + r) >> 1;
    pair<long long, int> best = {LLONG_MAX, -1};
    for (int k = optl; k <= min(mid, optr); k++) {
        best = min(best, {(k ? dp_before[k - 1] :
        ↪  0) + C(k, mid), k});
    }
    dp_cur[mid] = best.first;
    int opt = best.second;
    compute(l, mid - 1, optl, opt);
    compute(mid + 1, r, opt, optr);
}
int solve() {
    for (int i = 0; i < n; i++)
        dp_before[i] = C(0, i);

    for (int i = 1; i < m; i++) {
        compute(0, n - 1, 0, n - 1);
        dp_before = dp_cur;
    }
    return dp_before[n - 1];
}
```

2.4 Knapsack 01

```
/*
There are NN items, numbered 1,2,,N. For each i
    (1iN), Item i has a weight of wi and a value of
 ↪  vi.
Taro has decided to choose some of the N items and
    carry them home in a knapsack. The capacity of
 ↪  the knapsack is W, which means that the sum of
 ↪  the weights of items taken must be at most W.
Find the maximum possible sum of the values of
 ↪  items that Taro takes home.
*/
vector<vector<ll>> dp(110, vector<ll>(100000+10,
 ↪  -1));
```

```
ll fn(ll ind, ll wt, vector<pair<ll, ll>> &vc)
{
    if (ind < 0)
        return 0;
    if (wt < 0)
        return 0;
    if (dp[ind][wt] != -1)
        return dp[ind][wt];
    ll ans = fn(ind - 1, wt, vc);
    if (wt - vc[ind].first >= 0)
        ans = max(ans, fn(ind - 1, wt -
        ↪ vc[ind].first, vc) + vc[ind].second);
    return dp[ind][wt] = ans;
}
void solve()
{
    ll n, w;
    cin >> n >> w;
    vector<pair<ll, ll>> vc(n);
    for (int i = 0; i < n; i++)
    {
        cin >> vc[i].first >> vc[i].second;
    }
    cout << fn(n - 1, w, vc);
}
```

2.5 Knapsack 02

```
/*W <= 1e9*/
ll fn(ll ind, ll val, vector<pair<ll, ll>> &vc)
{
    if (val == 0)
        return 0;
    if (ind < 0)
        return 1e18;
    if (dp[ind][val] != -1)
        return dp[ind][val];
    ll ans = fn(ind - 1, val, vc);
    if (val - vc[ind].second >= 0)
        ans = min(ans, fn(ind - 1, val -
        ↪ vc[ind].second, vc) + vc[ind].first);
    return dp[ind][val] = ans;
}
void solve()
{
    ll n, w;
    cin >> n >> w;
    vector<pair<ll, ll>> vc(n);
    for (int i = 0; i < n; i++)
        cin >> vc[i].first >> vc[i].second;
    for (int i = 1e5 + 9; i >= 0; i--)
    {
        if (fn(n - 1, i, vc) <= w)
        {
            cout << i << "\n";
            return;
        }
    }
}
```

2.6 Knuth's Optomization

```
int solve() {
    int N;
    ... // read N and input
    int dp[N][N], opt[N][N];
    auto C = [&](int i, int j) {
        ... // Implement cost function C.
    };
    for (int i = 0; i < N; i++) {
        opt[i][i] = i;
        ... // Initialize dp[i][i] according to the
        ↪ problem
    }
    for (int i = N-2; i >= 0; i--) {
        for (int j = i+1; j < N; j++) {
            int mn = INT_MAX;
            int cost = C(i, j);
            for (int k = opt[i][j-1]; k <= min(j-1,
            ↪ opt[i+1][j]); k++) {
                if (mn >= dp[i][k] + dp[k+1][j] +
                ↪ cost) {
                    opt[i][j] = k;
                    mn = dp[i][k] + dp[k+1][j] +
                    ↪ cost;
                }
            }
            dp[i][j] = mn;
        }
    }
    cout << dp[0][N-1] << endl;
}
```

2.7 LCS

```
/*You are given strings ss and tt. Find one longest
↪ string that is a subsequence of both ss and
↪ tt.*/
void fn(ll i, ll j, vector<ll> & ans,
↪ vector<vector<ll>> &vc)
{
    if (vc[i][j] == 0)
        return;
    if (vc[i][j] > vc[i - 1][j] && vc[i][j] >
    ↪ vc[i][j - 1])
    {
        fn(i - 1, j - 1, ans, vc);
        ans.push_back(i);
    }
    else if (vc[i][j] == vc[i-1][j])
    {
        fn(i-1, j, ans, vc);
    }
    else{
        fn(i, j-1, ans, vc);
    }
}
void solve()
{
    string s1, s2;
    cin >> s1 >> s2;
    vector<vector<ll>> vc(s1.size() + 1,
    ↪ vector<ll>(s2.size() + 1, 0));
    for (int i = 0; i < s1.size(); i++)
    {
        for (int j = 0; j < s2.size(); j++)
        {
            if (s1[i] == s2[j])
            {
                vc[i + 1][j + 1] = 1 + vc[i][j];
            }
            else
            {
                vc[i + 1][j + 1] = max(vc[i][j +
                ↪ 1], vc[i + 1][j]);
            }
        }
    }
    vector<ll> ans;
    fn(s1.size(), s2.size(), ans, vc);
    for(auto ic: ans)
    cout << s1[ic-1];
}
```

2.8 Minimum sum Diff

```
/*You are given an integer array nums of 2 * n
    integers. You need to partition nums into two
    arrays of length n to minimize the absolute
↪ difference of the sums of the arrays. To
↪ partition nums, put each element of nums into
↪ one of the two arrays.
Return the minimum possible absolute difference.*/
int minimumDifference(vector<int> &nums)
{
    int n = nums.size(), sum = 0;
    sum = accumulate(nums.begin(), nums.end(), 0);
    ↪ // To find the total sum of the array
    int N = n / 2;
    ↪ // Divide it into two equals parts as
    ↪ length is even
    vector<vector<int>> left(N + 1), right(N + 1);
    ↪ // left array and right array
    // All possible sum in left and right part
    ↪ (Generating and storing using BIT-Masking)
    for (int mask = 0; mask < (1 << N); ++mask)
    { // (1<<n) means 2^n i.e we'll iterate through
    ↪ 0 to 2^n
        int idx = 0, l_sum = 0, r_sum = 0;
        for (int i = 0; i < N; ++i)
        {
            if (mask & (1 << i))
            { // To check if the bit is set or not
                idx++;
                l_sum += nums[i];
                r_sum += nums[i + N];
            }
        }
        left[idx].push_back(l_sum);
        right[idx].push_back(r_sum); // storing
    }
    for (int idx = 0; idx <= N; ++idx)
    {
        sort(right[idx].begin(), right[idx].end());
        ↪ // as we'll perform binary search on
        ↪ right so we have to sort it first
    }
    int res = min(abs(sum - 2 * left[N][0]),
    ↪ abs(sum - 2 * right[N][0])); // To get the
    ↪ minimum answer from the max sum from both
    ↪ array
    // iterating over left part
```

```
    for (int idx = 1; idx < N; ++idx)
    { // iterate from 1 so we dont have to include 0 and check for all value except last as we have alr considered it
        for (auto &a : left[idx])
        {
                // check the sum at each number position
            int b = (sum - 2 * a) / 2;
                // find the value to be minimized
            int rightidx = N - idx;
                // find the number value in right array
            auto &v = right[rightidx];
                // store the vector in v at right number position
            auto itr = lower_bound(v.begin(), v.end(), b); // binary search over right part

            if (itr != v.end())
                res = min(res, abs(sum - 2 * (a + (*itr)))); // if found in vector then only update otherwise continue
        }
    }
    return res;
}
```

2.9 Subset sum

```
/*O(sum * n)*/
#include <bits/stdc++.h>
using namespace std;
bool isSubsetSum(int set[], int n, int sum)
{
 // Base Cases
 if (sum == 0)
  return true;
 if (n == 0)
  return false;
 if (set[n - 1] > sum)
  return isSubsetSum(set, n - 1, sum);
 return isSubsetSum(set, n - 1, sum)
  || isSubsetSum(set, n - 1, sum - set[n - 1]);
}
int main()
{
 int set[] = { 3, 34, 4, 12, 5, 2 };
 int sum = 9;
 int n = sizeof(set) / sizeof(set[0]);
 if (isSubsetSum(set, n, sum) == true)
  cout << "Found a subset with given sum";
 else
  cout << "No subset with given sum";
 return 0;
}
```

3 Data Structure

3.1 BIT 2D with Range Update and Range Query

```
#include<bits/stdc++.h>
using namespace std;

const int N = 1010;

struct BIT2D {
  long long M[N][N][2], A[N][N][2];
  BIT2D() {
    memset(M, 0, sizeof M);
    memset(A, 0, sizeof A);
  }
  void upd2(long long t[N][N][2], int x, int y, long long mul, long long add) {
    for(int i = x; i < N; i += i & -i) {
      for(int j = y; j < N; j += j & -j) {
        t[i][j][0] += mul;
        t[i][j][1] += add;
      }
    }
  }
  void upd1(int x, int y1, int y2, long long mul, long long add) {
    upd2(M, x, y1, mul, -mul * (y1 - 1));
    upd2(M, x, y2, -mul, mul * y2);
    upd2(A, x, y1, add, -add * (y1 - 1));
    upd2(A, x, y2, -add, add * y2);
  }
  void upd(int x1, int y1, int x2, int y2, long long val) {
    upd1(x1, y1, y2, val, -val * (x1 - 1));
    upd1(x2, y1, y2, -val, val * x2);
  }
  long long query2(long long t[N][N][2], int x, int y) {
    long long mul = 0, add = 0;
    for(int i = y; i > 0; i -= i & -i) {
      mul += t[x][i][0];
      add += t[x][i][1];
    }
    return mul * y + add;
  }
  long long query1(int x, int y) {
    long long mul = 0, add = 0;
    for(int i = x; i > 0; i -= i & -i) {
      mul += query2(M, i, y);
      add += query2(A, i, y);
    }
    return mul * x + add;
  }
  long long query(int x1, int y1, int x2, int y2) {
    return query1(x2, y2) - query1(x1 - 1, y2) - query1(x2, y1 - 1) + query1(x1 - 1, y1 - 1);
  }
} t;
int main() {
  int n, m;
  cin >> n >> m;
  for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= m; j++) {
      int k;
      cin >> k;
      t.upd(i, j, i, j, k);
    }
  }
  int q;
  cin >> q;
  while(q--) {
    int ty, x1, y1, x2, y2;
    cin >> ty;
    if(ty == 1) {
      long long val;
      cin >> x1 >> y1 >> x2 >> y2 >> val;
      t.upd(x1, y1, x2, y2, val); // add val from top-left(x1, y1) to bottom-right (x2, y2);
    } else {
      cin >> x1 >> y1 >> x2 >> y2;
      cout << t.query(x1, y1, x2, y2) << '\n'; // output sum from top-left(x1, y1) to bottom-right (x2, y2);
    }
  }
  return 0;
}
```

3.2 BIT with Range Update and Range Query

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

struct BIT {
  long long M[N], A[N];
  BIT() {
    memset(M, 0, sizeof M);
    memset(A, 0, sizeof A);
  }
  void update(int i, long long mul, long long add) {
    while (i < N) {
      M[i] += mul;
      A[i] += add;
      i |= (i + 1);
    }
  }
  void upd(int l, int r, long long x) {
    update(l, x, -x * (l - 1));
    update(r, -x, x * r);
  }
  long long query(int i) {
    long long mul = 0, add = 0;
    int st = i;
    while (i >= 0) {
      mul += M[i];
      add += A[i];
      i = (i & (i + 1)) - 1;
    }
    return (mul * st + add);
  }
  long long query(int l, int r) {
    return query(r) - query(l - 1);
  }
} t;

int32_t main() {

  return 0;
}
```

3.3 BIT

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

template <class T>
struct BIT { //1-indexed
  int n; vector<T> t;
  BIT() {}
  BIT(int _n) {
    n = _n; t.assign(n + 1, 0);
  }
  T query(int i) {
    T ans = 0;
    for (; i >= 1; i -= (i & -i)) ans += t[i];
    return ans;
  }
  void upd(int i, T val) {
    if (i <= 0) return;
    for (; i <= n; i += (i & -i)) t[i] += val;
  }
  void upd(int l, int r, T val) {
    upd(l, val);
    upd(r + 1, -val);
  }
  T query(int l, int r) {
    return query(r) - query(l - 1);
  }
};
int32_t main() {

  return 0;
}
```

3.4 Dsu

```
void make_set(int v) {
    parent[v] = v;
    size[v] = 1;
}
int find_set(int v) {
    if (v == parent[v])
        return v;
    return parent[v] = find_set(parent[v]);
}
void union_sets(int a, int b) {
    a = find_set(a);
    b = find_set(b);
    if (a != b) {
        if (size[a] < size[b])
            swap(a, b);
        parent[b] = a;
        size[a] += size[b];
    }
}
```

3.5 LCA

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9, LG = 18;

vector<int> g[N];
int par[N][LG + 1], dep[N], sz[N];
void dfs(int u, int p = 0) {
    par[u][0] = p;
    dep[u] = dep[p] + 1;
    sz[u] = 1;
    for (int i = 1; i <= LG; i++) par[u][i] =
    ↪ par[par[u][i - 1]][i - 1];
    for (auto v: g[u]) if (v != p) {
        dfs(v, u);
        sz[u] += sz[v];
    }
}
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    for (int k = LG; k >= 0; k--) if
    ↪ (dep[par[u][k]] >= dep[v]) u = par[u][k];
    if (u == v) return u;
    for (int k = LG; k >= 0; k--) if (par[u][k] !=
    ↪ par[v][k]) u = par[u][k], v = par[v][k];
    return par[u][0];
}
int kth(int u, int k) {
    assert(k >= 0);
    for (int i = 0; i <= LG; i++) if (k & (1 << i))
    ↪ u = par[u][i];
    return u;
}
int dist(int u, int v) {
    int l = lca(u, v);
    return dep[u] + dep[v] - (dep[l] << 1);
}
//kth node from u to v, 0th node is u
int go(int u, int v, int k) {
    int l = lca(u, v);
    int d = dep[u] + dep[v] - (dep[l] << 1);
    assert(k <= d);
    if (dep[l] + k <= dep[u]) return kth(u, k);
    k -= dep[u] - dep[l];
    return kth(v, dep[v] - dep[l] - k);
}
int32_t main() {
    int n; cin >> n;
    for (int i = 1; i < n; i++) {
     int u, v; cin >> u >> v;
     g[u].push_back(v);
     g[v].push_back(u);
    }
    dfs(1);
    int q; cin >> q;
    while (q--) {
        int u, v; cin >> u >> v;
        cout << dist(u, v) << '\n';
    }
    return 0;
}
```

3.6 MOs Algorithm

```
#include<bits/stdc++.h>
using namespace std;

const int N = 1e6 + 9, B = 440;

struct query {
  int l, r, id;
  bool operator < (const query &x) const {
    if(l / B == x.l / B) return ((l / B) & 1) ? r >
    ↪ x.r : r < x.r;
    return l / B < x.l / B;
  }
} Q[N];
int cnt[N], a[N];
long long sum;
inline void add_left(int i) {
  int x = a[i];
  sum += 1LL * (cnt[x] + cnt[x] + 1) * x;
  ++cnt[x];
}
inline void add_right(int i) {
  int x = a[i];
  sum += 1LL * (cnt[x] + cnt[x] + 1) * x;
  ++cnt[x];
}
inline void rem_left(int i) {
  int x = a[i];
  sum -= 1LL * (cnt[x] + cnt[x] - 1) * x;
  --cnt[x];
}
inline void rem_right(int i) {
  int x = a[i];
  sum -= 1LL * (cnt[x] + cnt[x] - 1) * x;
  --cnt[x];
}
long long ans[N];
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);

  int n, q;
  cin >> n >> q;
  for(int i = 1; i <= n; i++) cin >> a[i];
  for(int i = 1; i <= q; i++) {
    cin >> Q[i].l >> Q[i].r;
    Q[i].id = i;
  }
  sort(Q + 1, Q + q + 1);
  int l = 1, r = 0;
  for(int i = 1; i <= q; i++) {
    int L = Q[i].l, R = Q[i].r;
    if(R < l) {
      while (l > L) add_left(--l);
      while (l < L) rem_left(l++);
      while (r < R) add_right(++r);
      while (r > R) rem_right(r--);
    } else {
      while (r < R) add_right(++r);
      while (r > R) rem_right(r--);
      while (l > L) add_left(--l);
      while (l < L) rem_left(l++);
    }
    ans[Q[i].id] = sum;
  }
  for(int i = 1; i <= q; i++) cout << ans[i] <<
  ↪ '\n';
  return 0;
}
```

3.7 MOs on tree

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

//unique elements on the path from u to v
vector<int> g[N];
int st[N], en[N], T, par[N][20], dep[N], id[N * 2];
void dfs(int u, int p = 0) {
  st[u] = ++T;
  id[T] = u;
  dep[u] = dep[p] + 1;
  par[u][0] = p;
  for(int k = 1; k < 20; k++) par[u][k] =
  ↪  par[par[u][k - 1]][k - 1];
  for(auto v : g[u]) if(v != p) dfs(v, u);
  en[u] = ++T;
  id[T] = u;
}
int lca(int u, int v) {
  if(dep[u] < dep[v]) swap(u, v);
  for(int k = 19; k >= 0; k--) if(dep[par[u][k]] >=
  ↪  dep[v]) u = par[u][k];
  if(u == v) return u;
  for(int k = 19; k >= 0; k--) if(par[u][k] !=
  ↪  par[v][k]) u = par[u][k], v = par[v][k];
  return par[u][0];
}

int cnt[N], a[N], ans;
inline void add(int u) {
  int x = a[u];
  if(cnt[x]++ == 0) ans++;
}
inline void rem(int u) {
  int x = a[u];
  if(--cnt[x] == 0) ans--;
}
bool vis[N];
inline void yo(int u) {
  if(!vis[u]) add(u);
  else rem(u);
  vis[u] ^= 1;
}

const int B = 320;
struct query {
  int l, r, id;
  bool operator < (const query &x) const {
    if(l / B == x.l / B) return r < x.r;
    return l / B < x.l / B;
  }
} Q[N];

int res[N];
int main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);

  int n, q;
  while(cin >> n >> q) {
    for(int i = 1; i <= n; i++) cin >> a[i];
    map<int, int> mp;
    for(int i = 1; i <= n; i++) {
      if(mp.find(a[i]) == mp.end()) mp[a[i]],
      ↪  mp[a[i]] = mp.size();
      a[i] = mp[a[i]];
    }
    for(int i = 1; i < n; i++) {
      int u, v;
      cin >> u >> v;
      g[u].push_back(v);
      g[v].push_back(u);
    }
    T = 0;
    dfs(1);
    for(int i = 1; i <= q; i++) {
      int u, v ;
      cin >> u >> v;
      if(st[u] > st[v]) swap(u, v);
      int lc = lca(u, v);
      if(lc == u) Q[i].l = st[u], Q[i].r = st[v];
      else Q[i].l = en[u], Q[i].r = st[v];
      Q[i].id = i;
    }
    sort(Q + 1, Q + q + 1);
    ans = 0;
    int l = 1, r = 0;
    for(int i = 1; i <= q; i++) {
      int L = Q[i].l, R = Q[i].r;
      if(R < l) {
        while (l > L) yo(id[--l]);
        while (l < L) yo(id[l++]);
        while (r < R) yo(id[++r]);
        while (r > R) yo(id[r--]);
      } else {
        while (r < R) yo(id[++r]);
        while (r > R) yo(id[r--]);
        while (l > L) yo(id[--l]);
        while (l < L) yo(id[l++]);
      }
      int u = id[l], v = id[r], lc = lca(u, v);
      if(lc != u && lc != v) yo(lc); //take care of
      ↪  the lca separately
      res[Q[i].id] = ans;
      if(lc != u && lc != v) yo(lc);
    }
    for(int i = 1; i <= q; i++) cout << res[i] <<
    ↪  '\n';
    for(int i = 0; i <= n; i++) {
      g[i].clear();
      vis[i] = cnt[i] = 0;
      for(int k = 0; k < 20; k++) par[i][k] = 0;
    }
  }
  return 0;
}
// https://www.spoj.com/problems/COT2/
```

3.8 MOs with Update

```
#include<bits/stdc++.h>
using namespace std;

const int N = 2e5 + 9;
const int B = 2500;

struct query {
  int l, r, t, id;
  bool operator < (const query &x) const {
    if(l / B == x.l / B) {
      if(r / B == x.r / B) return t < x.t;
      return r / B < x.r / B;
    }
    return l / B < x.l / B;
  }
} Q[N];
struct upd {
  int pos, old, cur;
} U[N];

int a[N];
int cnt[N], f[N], ans[N], l, r, t;
inline void add(int x) {
  f[cnt[x]]--, ++cnt[x], f[cnt[x]]++;
}
inline void del(int x) {
  f[cnt[x]]--, --cnt[x], f[cnt[x]]++;
}
inline void update(int pos, int x) {
  if (l <= pos && pos <= r) {
    add(x);
    del(a[pos]);
  }
  a[pos] = x;
}
map <int, int> mp;
int nxt = 0;
int get(int x) {
  return mp.count(x) ? mp[x] : mp[x] = ++nxt;
}
int main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);

  int n, q;
  cin >> n >> q;
  for (int i = 1; i <= n; i++) {
    cin >> a[i];
    a[i] = get(a[i]);
  }
  int nq = 0, nu = 0;
  for (int i = 1; i <= q; i++) {
    int ty, l, r;
    cin >> ty >> l >> r;
    if (ty == 1) ++nq, Q[nq] = {l, r, nu, nq};
    else ++nu, U[nu].pos = l, U[nu].old = a[l],
    ↪  a[l] = get(r), U[nu].cur = a[l];
  }
  sort(Q + 1, Q + nq + 1);
  t = nu, l = 1, r = 0;
  for (int i = 1; i <= nq; i++) {
    int L = Q[i].l, R = Q[i].r, T = Q[i].t;
    while(t < T) t++, update(U[t].pos, U[t].cur);
    while(t > T) update(U[t].pos, U[t].old), t--;
    if(R < l) {
      while(l > L) add(a[--l]);
      while(l < L) del(a[l++]);
      while(r < R) add(a[++r]);
      while(r > R) del(a[r--]);
    } else {
      while(r < R) add(a[++r]);
      while(r > R) del(a[r--]);
      while(l > L) add(a[--l]);
```

```
      while(l < L) del(a[l++]);
    }
    int cur = 1;
    while(f[cur] > 0) cur++;
    ans[Q[i].id] = cur;
  }
  for (int i = 1; i <= nq; i++) cout << ans[i] << '\n';
  return 0;
}
//https://codeforces.com/contest/940/problem/F
```

3.9 Ordered Set

```
#include<bits/stdc++.h>
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
using namespace std;

template <typename T> using o_set = tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
template <typename T, typename R> using o_map = tree<T, R, less<T>, rb_tree_tag, tree_order_statistics_node_update>;
int main() {
  int i, j, k, n, m;
  o_set<int>se;
  se.insert(1);
  se.insert(2);
  cout << *se.find_by_order(0) << endl; ///k th element
  cout << se.order_of_key(2) << endl; ///number of elements less than k
  o_map<int, int>mp;
  mp.insert({1, 10});
  mp.insert({2, 20});
  cout << mp.find_by_order(0)->second << endl; ///k th element
  cout << mp.order_of_key(2) << endl; ///number of first elements less than k
  return 0;
}
```

3.10 Segment Tree Lazy

```
#include<bits/stdc++.h>
using namespace std;

const int N = 5e5 + 9;
int a[N];
struct ST {
  #define lc (n << 1)
  #define rc ((n << 1) | 1)
  long long t[4 * N], lazy[4 * N];
  ST() {
    memset(t, 0, sizeof t);
    memset(lazy, 0, sizeof lazy);
  }
  inline void push(int n, int b, int e) {
    if (lazy[n] == 0) return;
    t[n] = t[n] + lazy[n] * (e - b + 1);
    if (b != e) {
      lazy[lc] = lazy[lc] + lazy[n];
      lazy[rc] = lazy[rc] + lazy[n];
    }
    lazy[n] = 0;
  }
  inline long long combine(long long a,long long b) {
    return a + b;
  }
  inline void pull(int n) {
    t[n] = t[lc] + t[rc];
  }
  void build(int n, int b, int e) {
    lazy[n] = 0;
    if (b == e) {
      t[n] = a[b];
      return;
    }
    int mid = (b + e) >> 1;
    build(lc, b, mid);
    build(rc, mid + 1, e);
    pull(n);
  }
  void upd(int n, int b, int e, int i, int j, long long v) {
    push(n, b, e);
    if (j < b || e < i) return;
    if (i <= b && e <= j) {
      lazy[n] = v; //set lazy
      push(n, b, e);
      return;
    }
    int mid = (b + e) >> 1;
    upd(lc, b, mid, i, j, v);
    upd(rc, mid + 1, e, i, j, v);
    pull(n);
  }
  long long query(int n, int b, int e, int i, int j) {
    push(n, b, e);
    if (i > e || b > j) return 0; //return null
    if (i <= b && e <= j) return t[n];
    int mid = (b + e) >> 1;
    return combine(query(lc, b, mid, i, j), query(rc, mid + 1, e, i, j));
  }
};
int32_t main() {

}
```

3.11 Segment Tree

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

int a[N];
struct ST {
  int t[4 * N];
  static const int inf = 1e9;
  ST() {
    memset(t, 0, sizeof t);
  }
  void build(int n, int b, int e) {
    if (b == e) {
      t[n] = a[b];
      return;
    }
    int mid = (b + e) >> 1, l = n << 1, r = l | 1;
    build(l, b, mid);
    build(r, mid + 1, e);
    t[n] = max(t[l], t[r]);
  }
  void upd(int n, int b, int e, int i, int x) {
    if (b > i || e < i) return;
    if (b == e && b == i) {
      t[n] = x;
      return;
    }
    int mid = (b + e) >> 1, l = n << 1, r = l | 1;
    upd(l, b, mid, i, x);
    upd(r, mid + 1, e, i, x);
    t[n] = max(t[l], t[r]);
  }
  int query(int n, int b, int e, int i, int j) {
    if (b > j || e < i) return -inf;
    if (b >= i && e <= j) return t[n];
    int mid = (b + e) >> 1, l = n << 1, r = l | 1;
    int L = query(l, b, mid, i, j);
    int R = query(r, mid + 1, e, i, j);
    return max(L, R);
  }
}t;

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);

  return 0;
}
```

3.12 Sparse table

```
int log2_floor(unsigned long i) {
    return std::bit_width(i) - 1;
}
K = log2_floor(MAXN) + 1;
int st[K + 1][MAXN];
std::copy(array.begin(), array.end(), st[0]);
for (int i = 1; i <= K; i++)
    for (int j = 0; j + (1 << i) <= N; j++)
        st[i][j] = min(st[i - 1][j], st[i - 1][j + (1 << (i - 1))]);
int i = log2_floor[R - L + 1];
int minimum = min(st[i][L], st[i][R - (1 << i) + 1]);
```

3.13 Trie

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

struct Trie {
  static const int B = 31;
  struct node {
```

```
    node* nxt[2];
    int sz;
    node() {
      nxt[0] = nxt[1] = NULL;
      sz = 0;
    }
  }*root;
  Trie() {
    root = new node();
  }
  void insert(int val) {
    node* cur = root;
    cur -> sz++;
    for (int i = B - 1; i >= 0; i--) {
      int b = val >> i & 1;
      if (cur -> nxt[b] == NULL) cur -> nxt[b] =
      ↪  new node();
      cur = cur -> nxt[b];
      cur -> sz++;
    }
  }
  int query(int x, int k) { // number of values
  ↪  s.t. val ^ x < k
    node* cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
      if (cur == NULL) break;
      int b1 = x >> i & 1, b2 = k >> i & 1;
      if (b2 == 1) {
        if (cur -> nxt[b1]) ans += cur -> nxt[b1]
        ↪  -> sz;
        cur = cur -> nxt[!b1];
      } else cur = cur -> nxt[b1];
    }
    return ans;
  }
  int get_max(int x) { // returns maximum of val ^ x
    node* cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
      int k = x >> i & 1;
      if (cur -> nxt[!k]) cur = cur -> nxt[!k], ans
      ↪  <<= 1, ans++;
      else cur = cur -> nxt[k], ans <<= 1;
    }
    return ans;
  }
  int get_min(int x) { // returns minimum of val ^ x
    node* cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
      int k = x >> i & 1;
      if (cur -> nxt[k]) cur = cur -> nxt[k], ans
      ↪  <<= 1;
      else cur = cur -> nxt[!k], ans <<= 1, ans++;
    }
    return ans;
  }
  void del(node* cur) {
    for (int i = 0; i < 2; i++) if (cur -> nxt[i])
    ↪  del(cur -> nxt[i]);
    delete(cur);
  }
} t;
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int n, k;
  cin >> n >> k;
  int cur = 0;
  long long ans = 1LL * n * (n + 1) / 2;
  t.insert(cur);
  for (int i = 0; i < n; i++) {
    int x;
    cin >> x;
    cur ^= x;
    ans -= t.query(cur, k);
    t.insert(cur);
  }
  cout << ans << '\n';
  return 0;
}
```

3.14 segment tree 2(sum)

4 Geometry

4.1 2D point

```
struct point2d {
    ftype x, y;
    point2d() {}
    point2d(ftype x, ftype y): x(x), y(y) {}
    point2d& operator+=(const point2d &t) {
        x += t.x;
        y += t.y;
        return *this;
    }
    point2d& operator-=(const point2d &t) {
        x -= t.x;
        y -= t.y;
        return *this;
    }
    point2d& operator*=(ftype t) {
        x *= t;
        y *= t;
        return *this;
    }
    point2d& operator/=(ftype t) {
        x /= t;
        y /= t;
        return *this;
    }
    point2d operator+(const point2d &t) const {
        return point2d(*this) += t;
    }
    point2d operator-(const point2d &t) const {
        return point2d(*this) -= t;
    }
    point2d operator*(ftype t) const {
        return point2d(*this) *= t;
    }
    point2d operator/(ftype t) const {
        return point2d(*this) /= t;
    }
};
point2d operator*(ftype a, point2d b) {
    return b * a;
}
```

4.2 3D point

```
struct point3d {
    ftype x, y, z;
    point3d() {}
    point3d(ftype x, ftype y, ftype z): x(x), y(y),
    ↪  z(z) {}
    point3d& operator+=(const point3d &t) {
        x += t.x;
        y += t.y;
        z += t.z;
        return *this;
    }
    point3d& operator-=(const point3d &t) {
        x -= t.x;
        y -= t.y;
        z -= t.z;
        return *this;
    }
    point3d& operator*=(ftype t) {
        x *= t;
        y *= t;
        z *= t;
        return *this;
    }
    point3d& operator/=(ftype t) {
        x /= t;
        y /= t;
        z /= t;
        return *this;
    }
    point3d operator+(const point3d &t) const {
        return point3d(*this) += t;
    }
    point3d operator-(const point3d &t) const {
        return point3d(*this) -= t;
    }
    point3d operator*(ftype t) const {
        return point3d(*this) *= t;
    }
    point3d operator/(ftype t) const {
        return point3d(*this) /= t;
    }
};
point3d operator*(ftype a, point3d b) {
    return b * a;
}
```

4.3 Area of polygon

```
double area(const vector<point>& fig) {
    double res = 0;
    for (unsigned i = 0; i < fig.size(); i++) {
        point p = i ? fig[i - 1] : fig.back();
        point q = fig[i];
        res += (p.x - q.x) * (p.y + q.y);
    }
    return fabs(res) / 2;
}
```

4.4 Check Intersection

```
struct pt {
    long long x, y;
```

```
    pt() {}
    pt(long long _x, long long _y) : x(_x), y(_y) {}
    pt operator-(const pt& p) const { return pt(x - p.x, y - p.y); }
    long long cross(const pt& p) const { return x * p.y - y * p.x; }
    long long cross(const pt& a, const pt& b) const { return (a - *this).cross(b - *this); }
};
int sgn(const long long& x) { return x >= 0 ? x ? 1 : 0 : -1; }
bool inter1(long long a, long long b, long long c, long long d) {
    if (a > b)
        swap(a, b);
    if (c > d)
        swap(c, d);
    return max(a, c) <= min(b, d);
}
bool check_inter(const pt& a, const pt& b, const pt& c, const pt& d) {
    if (c.cross(a, d) == 0 && c.cross(b, d) == 0)
        return inter1(a.x, b.x, c.x, d.x) && inter1(a.y, b.y, c.y, d.y);
    return sgn(a.cross(b, c)) != sgn(a.cross(b, d)) &&
            sgn(c.cross(d, a)) != sgn(c.cross(d, b));
}
```

4.5 Circle line intersection

```
double r, a, b, c; // given as input
double x0 = -a*c/(a*a+b*b), y0 = -b*c/(a*a+b*b);
if (c*c > r*r*(a*a+b*b)+EPS)
    puts ("no points");
else if (abs (c*c - r*r*(a*a+b*b)) < EPS) {
    puts ("1 point");
    cout << x0 << ' ' << y0 << '\n';
}
else {
    double d = r*r - c*c/(a*a+b*b);
    double mult = sqrt (d / (a*a+b*b));
    double ax, ay, bx, by;
    ax = x0 + b * mult;
    bx = x0 - b * mult;
    ay = y0 - a * mult;
    by = y0 + a * mult;
    puts ("2 points");
    cout << ax << ' ' << ay << '\n' << bx << ' ' << by << '\n';
}
```

4.6 Common tengents of circle

```
struct pt {
    double x, y;

    pt operator- (pt p) {
        pt res = { x-p.x, y-p.y };
        return res;
    }
};
struct circle : pt {
    double r;
};
struct line {
    double a, b, c;
};
const double EPS = 1E-9;
double sqr (double a) {
    return a * a;
}
void tangents (pt c, double r1, double r2, vector<line> & ans) {
    double r = r2 - r1;
    double z = sqr(c.x) + sqr(c.y);
    double d = z - sqr(r);
    if (d < -EPS)  return;
    d = sqrt (abs (d));
    line l;
    l.a = (c.x * r + c.y * d) / z;
    l.b = (c.y * r - c.x * d) / z;
    l.c = r1;
    ans.push_back (l);
}
vector<line> tangents (circle a, circle b) {
    vector<line> ans;
    for (int i=-1; i<=1; i+=2)
        for (int j=-1; j<=1; j+=2)
            tangents (b-a, a.r*i, b.r*j, ans);
    for (size_t i=0; i<ans.size(); ++i)
        ans[i].c -= ans[i].a * a.x + ans[i].b * a.y;
    return ans;
}
```

4.7 Convex hall

```
struct pt {
    double x, y;
};
int orientation(pt a, pt b, pt c) {
    double v = a.x*(b.y-c.y)+b.x*(c.y-a.y)+c.x*(a.y-b.y);
    if (v < 0) return -1; // clockwise
    if (v > 0) return +1; // counter-clockwise
    return 0;
}
bool cw(pt a, pt b, pt c, bool include_collinear) {
    int o = orientation(a, b, c);
    return o < 0 || (include_collinear && o == 0);
}
bool ccw(pt a, pt b, pt c, bool include_collinear) {
    int o = orientation(a, b, c);
    return o > 0 || (include_collinear && o == 0);
}
void convex_hull(vector<pt>& a, bool include_collinear = false) {
    if (a.size() == 1)
        return;

    sort(a.begin(), a.end(), [](pt a, pt b) {
        return make_pair(a.x, a.y) < make_pair(b.x, b.y);
    });
    pt p1 = a[0], p2 = a.back();
    vector<pt> up, down;
    up.push_back(p1);
    down.push_back(p1);
    for (int i = 1; i < (int)a.size(); i++) {
        if (i == a.size() - 1 || cw(p1, a[i], p2, include_collinear)) {
            while (up.size() >= 2 &&
                !cw(up[up.size()-2], up[up.size()-1], a[i], include_collinear))
                up.pop_back();
            up.push_back(a[i]);
        }
        if (i == a.size() - 1 || ccw(p1, a[i], p2, include_collinear)) {
            while (down.size() >= 2 &&
                !ccw(down[down.size()-2], down[down.size()-1], a[i], include_collinear))
                down.pop_back();
            down.push_back(a[i]);
        }
    }

    if (include_collinear && up.size() == a.size()) {
        reverse(a.begin(), a.end());
        return;
    }
    a.clear();
    for (int i = 0; i < (int)up.size(); i++)
        a.push_back(up[i]);
    for (int i = down.size() - 2; i > 0; i--)
        a.push_back(down[i]);
}
```

4.8 Intersection of segments

```
const double EPS = 1E-9;
struct pt {
    double x, y;

    bool operator<(const pt& p) const
    {
        return x < p.x - EPS || (abs(x - p.x) < EPS && y < p.y - EPS);
    }
};
struct line {
    double a, b, c;

    line() {}
    line(pt p, pt q)
    {
        a = p.y - q.y;
        b = q.x - p.x;
        c = -a * p.x - b * p.y;
        norm();
    }

    void norm()
    {
        double z = sqrt(a * a + b * b);
        if (abs(z) > EPS)
            a /= z, b /= z, c /= z;
    }

    double dist(pt p) const { return a * p.x + b * p.y + c; }
```

```
};
double det(double a, double b, double c, double d)
{
    return a * d - b * c;
}
inline bool betw(double l, double r, double x)
{
    return min(l, r) <= x + EPS && x <= max(l, r) +
    ↪ EPS;
}
inline bool intersect_1d(double a, double b, double
↪ c, double d)
{
    if (a > b)
        swap(a, b);
    if (c > d)
        swap(c, d);
    return max(a, c) <= min(b, d) + EPS;
}
bool intersect(pt a, pt b, pt c, pt d, pt& left,
↪ pt& right)
{
    if (!intersect_1d(a.x, b.x, c.x, d.x) ||
    ↪ !intersect_1d(a.y, b.y, c.y, d.y))
        return false;
    line m(a, b);
    line n(c, d);
    double zn = det(m.a, m.b, n.a, n.b);
    if (abs(zn) < EPS) {
        if (abs(m.dist(c)) > EPS || abs(n.dist(a))
        ↪ > EPS)
            return false;
        if (b < a)
            swap(a, b);
        if (d < c)
            swap(c, d);
        left = max(a, c);
        right = min(b, d);
        return true;
    } else {
        left.x = right.x = -det(m.c, m.b, n.c, n.b)
        ↪ / zn;
        left.y = right.y = -det(m.a, m.c, n.a, n.c)
        ↪ / zn;
        return betw(a.x, b.x, left.x) && betw(a.y,
        ↪ b.y, left.y) &&
            betw(c.x, d.x, left.x) && betw(c.y,
            ↪ d.y, left.y);
    }
}
```

4.9 Line intersection

```
struct pt {
    double x, y;
};
struct line {
    double a, b, c;
};
const double EPS = 1e-9;
double det(double a, double b, double c, double d) {
    return a*d - b*c;
}
bool intersect(line m, line n, pt & res) {
    double zn = det(m.a, m.b, n.a, n.b);
    if (abs(zn) < EPS)
        return false;
    res.x = -det(m.c, m.b, n.c, n.b) / zn;
    res.y = -det(m.a, m.c, n.a, n.c) / zn;
    return true;
}
bool parallel(line m, line n) {
    return abs(det(m.a, m.b, n.a, n.b)) < EPS;
}
bool equivalent(line m, line n) {
    return abs(det(m.a, m.b, n.a, n.b)) < EPS
        && abs(det(m.a, m.c, n.a, n.c)) < EPS
        && abs(det(m.b, m.c, n.b, n.c)) < EPS;
}
```

4.10 Oriented area of a triangle

```
int signed_area_parallelogram(point2d p1, point2d
↪ p2, point2d p3) {
    return cross(p2 - p1, p3 - p2);
}
double triangle_area(point2d p1, point2d p2,
↪ point2d p3) {
    return abs(signed_area_parallelogram(p1, p2,
    ↪ p3)) / 2.0;
}
bool clockwise(point2d p1, point2d p2, point2d p3) {
    return signed_area_parallelogram(p1, p2, p3) <
    ↪ 0;
}
bool counter_clockwise(point2d p1, point2d p2,
↪ point2d p3) {
    return signed_area_parallelogram(p1, p2, p3) >
    ↪ 0;
}
```

4.11 check if belongs to convex poly

```
struct pt {
    long long x, y;
    pt() {}
    pt(long long _x, long long _y) : x(_x), y(_y) {}
    pt operator+(const pt &p) const { return pt(x +
    ↪ p.x, y + p.y); }
    pt operator-(const pt &p) const { return pt(x -
    ↪ p.x, y - p.y); }
    long long cross(const pt &p) const { return x *
    ↪ p.y - y * p.x; }
    long long dot(const pt &p) const { return x *
    ↪ p.x + y * p.y; }
    long long cross(const pt &a, const pt &b) const
    ↪ { return (a - *this).cross(b - *this); }
    long long dot(const pt &a, const pt &b) const {
    ↪ return (a - *this).dot(b - *this); }
    long long sqrLen() const { return
    ↪ this->dot(*this); }
};
bool lexComp(const pt &l, const pt &r) {
    return l.x < r.x || (l.x == r.x && l.y < r.y);
}
int sgn(long long val) { return val > 0 ? 1 : (val
↪ == 0 ? 0 : -1); }
vector<pt> seq;
pt translation;
int n;
bool pointInTriangle(pt a, pt b, pt c, pt point) {
    long long s1 = abs(a.cross(b, c));
    long long s2 = abs(point.cross(a, b)) +
    ↪ abs(point.cross(b, c)) + abs(point.cross(c,
    ↪ a));
    return s1 == s2;
}
void prepare(vector<pt> &points) {
    n = points.size();
    int pos = 0;
    for (int i = 1; i < n; i++) {
        if (lexComp(points[i], points[pos]))
            pos = i;
    }
    rotate(points.begin(), points.begin() + pos,
    ↪ points.end());
    n--;
    seq.resize(n);
    for (int i = 0; i < n; i++)
        seq[i] = points[i + 1] - points[0];
    translation = points[0];
}
bool pointInConvexPolygon(pt point) {
    point = point - translation;
    if (seq[0].cross(point) != 0 &&
            sgn(seq[0].cross(point)) !=
            ↪ sgn(seq[0].cross(seq[n - 1])))
        return false;
    if (seq[n - 1].cross(point) != 0 &&
            sgn(seq[n - 1].cross(point)) !=
            ↪ sgn(seq[n - 1].cross(seq[0])))
        return false;

    if (seq[0].cross(point) == 0)
        return seq[0].sqrLen() >= point.sqrLen();

    int l = 0, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) / 2;
        int pos = mid;
        if (seq[pos].cross(point) >= 0)
            l = mid;
        else
            r = mid;
    }
    int pos = l;
    return pointInTriangle(seq[pos], seq[pos + 1],
    ↪ pt(0, 0), point);
}
```

4.12 common type

```
ftype dot(point2d a, point2d b) {
    return a.x * b.x + a.y * b.y;
}
ftype dot(point3d a, point3d b) {
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
ftype norm(point2d a) {
    return dot(a, a);
}
double abs(point2d a) {
```

```
    return sqrt(norm(a));
}
double proj(point2d a, point2d b) {
    return dot(a, b) / abs(b);
}
double angle(point2d a, point2d b) {
    return acos(dot(a, b) / abs(a) / abs(b));
}
point3d cross(point3d a, point3d b) {
    return point3d(a.y * b.z - a.z * b.y,
                   a.z * b.x - a.x * b.z,
                   a.x * b.y - a.y * b.x);
}
ftype triple(point3d a, point3d b, point3d c) {
    return dot(a, cross(b, c));
}
ftype cross(point2d a, point2d b) {
    return a.x * b.y - a.y * b.x;
}
point2d intersect(point2d a1, point2d d1, point2d
 ↪  a2, point2d d2) {
    return a1 + cross(a2 - a1, d2) / cross(d1, d2)
     ↪  * d1;
}
point3d intersect(point3d a1, point3d n1, point3d
 ↪  a2, point3d n2, point3d a3, point3d n3) {
    point3d x(n1.x, n2.x, n3.x);
    point3d y(n1.y, n2.y, n3.y);
    point3d z(n1.z, n2.z, n3.z);
    point3d d(dot(a1, n1), dot(a2, n2), dot(a3,
     ↪  n3));
    return point3d(triple(d, y, z),
                   triple(x, d, z),
                   triple(x, y, d)) / triple(n1,
                    ↪  n2, n3);
}
```

5 Graph

5.1 Articulation Points

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

int T, low[N], dis[N], art[N];
vector<int> g[N];
void dfs(int u, int pre = 0) {
  low[u] = dis[u] = ++T;
  int child = 0;
  for(auto v: g[u]) {
    if(!dis[v]) {
      dfs(v, u);
      low[u] = min(low[u], low[v]);
      if(low[v] >= dis[u] && pre != 0) art[u] = 1;
      ++child;
    }
    else if(v != pre) low[u] = min(low[u], dis[v]);
  }
  if(pre == 0 && child > 1) art[u] = 1;
}
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  while(1){
    int n, m; cin >> n >> m;
    if(!n) break;
    while(m--) {
      int u, v; cin >> u >> v;
      g[u].push_back(v);
      g[v].push_back(u);
    }
    dfs(1);
    int ans = 0;
    for(int i = 1; i <= n; i++) ans += art[i];
    cout << ans << '\n';
    T = 0; for(int i = 1; i <= n; i++) low[i] =
     ↪  dis[i] = art[i] = 0, g[i].clear();
  }
  return 0;
}
```

5.2 Bellman Ford

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

struct st {
  int a, b, cost;
} e[N];
const int INF = 2e9;
int32_t main() {
  int n, m;
  cin >> n >> m;
  for(int i = 0; i < m; i++) cin >> e[i].a >>
   ↪  e[i].b >> e[i].cost;
  int s;
  cin >> s;//is there any negative cycle which is
   ↪  reachable from s?
  vector<int> d (n, INF);//for finding any
   ↪  cycle(not necessarily from s) set d[i] = 0
   ↪  for all i
  d[s] = 0;
  vector<int> p (n, -1);
  int x;
  for (int i=0; i<n; ++i) {
    x = -1;
    for (int j=0; j<m; ++j) {
      if (d[e[j].a] < INF) {
        if (d[e[j].b] > d[e[j].a] + e[j].cost) {
          d[e[j].b] = max (-INF, d[e[j].a] +
           ↪  e[j].cost);//for overflow
          p[e[j].b] = e[j].a;
          x = e[j].b;
        }
      }
    }
  }
  if (x == -1) cout << "No negative cycle from "<<s;
  else {
    int y = x; //x can be on any cycle or reachable
     ↪  from some cycle
    for (int i=0; i<n; ++i) y = p[y];

    vector<int> path;
    for (int cur=y; ; cur=p[cur]) {
      path.push_back (cur);
      if (cur == y && path.size() > 1) break;
    }
    reverse (path.begin(), path.end());

    cout << "Negative cycle: ";
    for (int i=0; i<path.size(); ++i) cout <<
     ↪  path[i] << ' ';
  }
  return 0;
}
```

5.3 Directed MST

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

const long long inf = 1e18;

template<typename T> struct PQ {
  long long sum = 0;
  priority_queue<T, vector<T>, greater<T>> Q;
  void push(T x) { x.w -= sum; Q.push(x); }
  T pop() { auto ans = Q.top(); Q.pop(); ans.w +=
   ↪  sum; return ans; }
  int size() { return Q.size(); }
  void add(long long x) { sum += x; }
  void merge(PQ &&x) {
    if (size() < x.size()) swap(sum, x.sum),
     ↪  swap(Q, x.Q);
    while (x.size()) {
      auto tmp = x.pop();
      tmp.w -= sum;
      Q.push(tmp);
    }
  }
};
struct edge {
  int u, v; long long w;
  bool operator > (const edge &rhs) const { return
   ↪  w > rhs.w; }
};
struct DSU {
  vector<int> par;
  DSU (int n) : par(n, -1) {}
  int root(int i) { return par[i] < 0 ? i : par[i]
   ↪  = root(par[i]); }
  void set_par(int c, int p) { par[c] = p; }
};
// returns parents of each vertex
// each edge should be distinct
// it assumes that a solution exists (all vertices
 ↪  are reachable from root)
// 0 indexed
// Takes ~300ms for n = 2e5
vector<int> DMST(int n, int root, const
 ↪  vector<edge> &edges) {
  vector<int> u(2 * n - 1, -1), par(2 * n - 1, -1);
  edge par_edge[2 * n - 1];
  vector<int> child[2 * n - 1];

  PQ<edge> Q[2 * n - 1];
  for (auto e : edges) Q[e.v].push(e);
  for (int i = 0; i < n; i++) Q[(i + 1) %
   ↪  n].push({i, (i + 1) % n, inf});
```

```
  int super = n;
  DSU dsu(2 * n - 1);
  int head = 0;
  while (Q[head].size()) {
    auto x = Q[head].pop();
    int nxt_root = dsu.root(x.u);
    if (nxt_root == head) continue;
    u[head] = nxt_root;
    par_edge[head] = x;
    if (u[nxt_root] == -1) head = nxt_root;
    else {
      int j = nxt_root;
      do {
        Q[j].add(-par_edge[j].w);
        Q[super].merge(move(Q[j]));
        assert(u[j] != -1);
        child[super].push_back(j);
        j = dsu.root(u[j]);
      } while (j != nxt_root);
      for (auto u : child[super]) par[u] = super,
      ↪ dsu.set_par(u, super);
      head = super++;
    }
  }
  vector<int> res(2 * n - 1, -1);
  queue<int> q; q.push(root);
  while (q.size()) {
    int u = q.front();
    q.pop();
    while (par[u] != -1) {
      for (auto v : child[par[u]]) {
        if (v != u) {
          res[par_edge[v].v] = par_edge[v].u;
          q.push(par_edge[v].v);
          par[v] = -1;
        }
      }
      u = par[u];
    }
  }
  res[root] = root; res.resize(n);
  return res;
}
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int n, m, root; cin >> n >> m >> root;
  vector<edge> edges(m);
  for (auto &e : edges) cin >> e.u >> e.v >> e.w;
  auto res = DMST(n, root, edges);

  unordered_map<int, int> W[n];
  for (auto u : edges) W[u.v][u.u] = u.w;

  long long ans = 0;
  for (int i = 0; i < n; i++) if (i != root) ans +=
  ↪ W[i][res[i]];
  cout << ans << '\n';
  for (auto x : res) cout << x << ' '; cout << '\n';
  return 0;
}
// https://judge.yosupo.jp/problem/directedmst
// http://www.cs.tau.ac.il/~zwick/grad-algo-13/dire
↪ cted-mst.pdf
```

5.4 Floyd Warshall

```
#include<bits/stdc++.h>
using namespace std;

const int N = 105;
int d[N][N];
int main() {
  int n = 10;
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
      if (i != j) {
        d[i][j] = 1e9;
      }
    }
  }
  for (int k = 1; k <= n; ++k) {
    for (int i = 1; i <= n; ++i) {
      for (int j = 1; j <= n; ++j) {
        d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
      }
    }
  }
  return 0;
}
```

5.5 Johnson's Algorithm

```
#include<bits/stdc++.h>
using namespace std;

const int N = 1010;

// Johnson's algorithm for all pair shortest paths
↪ with negative edges
// Complexity: O(N * M) + O(N * M * log(N))

const long long INF = 1e18;

struct edge {
  int u, v;
  long long w;
  edge(){}
  edge(int u, int v, long long w) : u(u), v(v),
  ↪ w(w){}
};

bool bellman_ford(int n, int s, vector <struct
↪ edge> ed, vector <long long> &d){
  d[s] = 0;
  for (int i = 0; i <= n; i++){
    int flag = 0;
    for (auto e: ed){
      if (d[e.u] + e.w < d[e.v]){
        flag = 1;
        d[e.v] = d[e.u] + e.w;
      }
    }
    if (flag == 0) return 1;
  }
  return 0;
}

vector <long long> dijkstra(int n, int s, vector
↪ <struct edge> ed, vector <long long> &h){
  vector <pair<int, long long> > g[n + 1];
  priority_queue<pair<long long, int>,
  ↪ vector<pair<long long, int>>,
  ↪ greater<pair<long long, int>>> q;
  for (auto e: ed){
    g[e.u].push_back({e.v, e.w + h[e.u] - h[e.v]});
  }
  vector <long long> d(n + 1, INF);vector<bool>
  ↪ vis(n + 1, 0);
  q.push({0, s});
  d[s] = 0;
  while(!q.empty()){
    auto x = q.top(); q.pop();
    int u = x.second;
    if(vis[u]) continue; vis[u] = 1;
    for(auto e: g[u]){
      int v = e.first; long long w = e.second;
      if(d[u] + w < d[v]){
        d[v] = d[u] + w; q.push({d[v], v});
      }
    }
  }
  for(int i = 1; i <= n; i++) if(d[i] < INF) d[i]
  ↪ -= h[s] - h[i];
  return d;
}

long long ans[N][N];
int johnson(int n, vector <struct edge> ed){
  vector <long long> potential(n + 1, INF);
  for (int i = 1; i <= n; i++) ed.push_back(edge(0,
  ↪ i, 0));
  if(bellman_ford(n, 0, ed, potential) == 0) return
  ↪ 0;
  cout << "yes\n\n";
  for(int i = 1; i <= n; i++){
    if(i > 1) cout << ' ';
    cout << potential[i];
  }
  cout << ' ' << potential[0] << "\n\n";
  for (int i = 1; i <= n; i++) ed.pop_back();
  for (int i = 1; i <= n; i++){
    vector <long long> d = dijkstra(n, i, ed,
    ↪ potential);
    for (int j = 1; j <= n; j++) ans[i][j] = d[j];
  }
  return 1;
}
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int t, cs = 0; cin >> t;
  while(t--){
    int n , m; cin >> n >> m;
    vector <edge> ed;
    while(m--){
      int u, v, w; cin >> u >> v >> w;
      ed.push_back({u, v, w});
    }
    cout << "graph " << ++cs << ' ';
    int ok = johnson(n, ed);
    if(!ok){
      cout << "no\n";
      continue;
```

```
    }
    for (int i = 1; i <= n; i++){
      for (int j = 1; j <= n; j++){
        if(j > 1) cout << ' ';
        if(ans[i][j] < INF) cout << ans[i][j];
        else cout << "#";
      }
      cout << '\n';
    }
  }
  return 0;
}
// https://www.spoj.com/problems/JHNSN/
```

5.6 LCA

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9, LG = 18;

vector<int> g[N];
int par[N][LG + 1], dep[N], sz[N];
void dfs(int u, int p = 0) {
  par[u][0] = p;
  dep[u] = dep[p] + 1;
  sz[u] = 1;
  for (int i = 1; i <= LG; i++) par[u][i] =
  ↪ par[par[u][i - 1]][i - 1];
  for (auto v: g[u]) if (v != p) {
    dfs(v, u);
    sz[u] += sz[v];
  }
}
int lca(int u, int v) {
  if (dep[u] < dep[v]) swap(u, v);
  for (int k = LG; k >= 0; k--) if (dep[par[u][k]]
  ↪ >= dep[v]) u = par[u][k];
  if (u == v) return u;
  for (int k = LG; k >= 0; k--) if (par[u][k] !=
  ↪ par[v][k]) u = par[u][k], v = par[v][k];
  return par[u][0];
}
int kth(int u, int k) {
  assert(k >= 0);
  for (int i = 0; i <= LG; i++) if (k & (1 << i)) u
  ↪ = par[u][i];
  return u;
}
int dist(int u, int v) {
  int l = lca(u, v);
  return dep[u] + dep[v] - (dep[l] << 1);
}
//kth node from u to v, 0th node is u
int go(int u, int v, int k) {
  int l = lca(u, v);
  int d = dep[u] + dep[v] - (dep[l] << 1);
  assert(k <= d);
  if (dep[l] + k <= dep[u]) return kth(u, k);
  k -= dep[u] - dep[l];
  return kth(v, dep[v] - dep[l] - k);
}
int32_t main() {
  int n; cin >> n;
  for (int i = 1; i < n; i++) {
    int u, v; cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u);
  }
  dfs(1);
  int q; cin >> q;
  while (q--) {
    int u, v; cin >> u >> v;
    cout << dist(u, v) << '\n';
  }
  return 0;
}
```

5.7 Prim's MST

```
#include<bits/stdc++.h>
using namespace std;

const int N = 2020;
int g[N][N], w[N], to[N], selected[N];
long long Prims(int n, vector< pair<int, int> >
↪ &edges) {
  long long ans = 0;
  for(int i = 1; i <= n; i++) w[i] = 1e9,
  ↪ selected[i] = 0, to[i] = -1;
  w[1] = 0;
  for(int i = 1; i <= n; i++) {
    int u = -1;
    for(int j = 1; j <= n; j++) if(!selected[j] &&
    ↪ (u == -1 || w[j] < w[u])) u = j;
    if (w[u] == 1e9) return - 1; //NO MST
    selected[u] = 1;
    ans += w[u];
    if(to[u] != -1) edges.emplace_back(u, to[u]);
    ↪ //order of the edges may be changed
    for(int v = 1; v <= n; v++) if(g[u][v] < w[v])
    ↪ w[v] = g[u][v], to[v] = u;
  }
  return ans;
}
string s[N];
int main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);

  int n, m; cin >> n >> m;
  for(int i = 1; i <= n; i++) for(int j = 1; j <=
  ↪ n; j++) g[i][j] = 1e9;
  for(int i = 1; i <= n; i++) cin >> s[i];
  for(int i = 1; i <= n; i++){
    for(int j = i + 1; j <= n; j++){
      int w = 0;
      for(int k = 0; k < m; k++) w = max(w,
      ↪ (int)abs(s[i][k] - s[j][k]));
      g[i][j] = min(g[i][j], w);
      g[j][i] = min(g[j][i], w);
    }
  }
  vector< pair<int, int> > ed;
  long long ans = Prims(n, ed);
  int res = 0; for(auto e: ed) res = max(res,
  ↪ g[e.first][e.second]);
  cout << res << '\n';
  return 0;
}
```

```
/*
https://www.codechef.com/ICL2016/problems/ICL16A
*/
```

5.8 SCC

```
#include<bits/stdc++.h>
using namespace std;
const int N = 3e5 + 9;

// given a directed graph return the minimum number
↪ of edges to be added so that the whole graph
↪ become an SCC
bool vis[N];
vector<int> g[N], r[N], G[N], vec; //G is the
↪ condensed graph
void dfs1(int u) {
  vis[u] = 1;
  for(auto v: g[u]) if(!vis[v]) dfs1(v);
  vec.push_back(u);
}

vector<int> comp;
void dfs2(int u) {
  comp.push_back(u);
  vis[u] = 1;
  for(auto v: r[u]) if(!vis[v]) dfs2(v);
}

int idx[N], in[N], out[N];
int main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);

  int n, m;
  cin >> n >> m;
  for(int i = 1; i <= m; i++) {
    int u, v;
    cin >> u >> v;
    g[u].push_back(v);
    r[v].push_back(u);
  }
  for(int i = 1; i <= n; i++) if(!vis[i]) dfs1(i);
  reverse(vec.begin(), vec.end());
  memset(vis, 0, sizeof vis);
  int scc = 0;
  for(auto u: vec) {
    if(!vis[u]) {
      comp.clear();
      dfs2(u);
      scc++;
      for(auto x: comp) idx[x]=scc;
    }
  }
  for(int u = 1; u <= n; u++) {
    for(auto v: g[u]) {
      if(idx[u] != idx[v]) {
        in[idx[v]]++, out[idx[u]]++;
        G[idx[u]].push_back(idx[v]);
      }
    }
  }
  int needed_in=0, needed_out=0;
  for(int i = 1; i <= scc; i++) {
```

```
    if(!in[i]) needed_in++;
    if(!out[i]) needed_out++;
  }
  int ans = max(needed_in, needed_out);
  if(scc == 1) ans = 0;
  cout << ans << '\n';
  return 0;
}
```

5.9 SPFA

```
#include<bits/stdc++.h>
using namespace std;

const int N = 5050;

vector<pair<int, int>> g[N];
bool vis[N];
long long d[N];
bool spfa(int u) {
  vis[u] = 1;
  for(auto e: g[u]) {
    int w = e.second, v = e.first;
    if(d[u] + w < d[v]) {
      if(vis[v]) return 0;
      d[v] = d[u] + w;
      if(!spfa(v)) return 0;
    }
  }
  vis[u] = 0;
  return 1;
}

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);

  int t;
  cin >> t;
  while(t--) {
    int n, m;
    cin >> n >> m;
    for(int i = 0; i <= n; i++) g[i].clear();
    int ans = 1e9;
    for(int i = 1; i <= m; i++) {
      int u, v, w; cin >> u >> v >> w;
      g[u].push_back({v, w});
      ans = min(ans, w);
    }
    if(ans >= 0) {
      cout << ans << '\n';
      continue;
    }
    for(int i = 1; i <= n; i++) g[0].push_back({i,
    ↪  0});
    for(int i = 0; i <= n; i++) d[i] = 1e10, vis[i]
    ↪  = 0;
    int s = 0;
    d[s] = 0;
    if(spfa(s) == 0) cout << "-inf\n"; // negative
    ↪  cycle
    else {
      long long ans = d[1];
      for(int i = 1; i <= n; i++) ans= min(ans,
      ↪  d[i]);
      cout << ans << '\n';
    }
  }
  return 0;
}
// https://codeforces.com/gym/101498/problem/L
```

6 Linear Algebra

6.1 Determinant

```
const double EPS = 1E-9;
int n;
vector<vector<double>> a(n, vector<double>(n));
double det = 1;
for (int i = 0; i < n; ++i)
{
    int k = i;
    for (int j = i + 1; j < n; ++j)
        if (abs(a[j][i]) > abs(a[k][i]))
            k = j;
    if (abs(a[k][i]) < EPS)
    {
        det = 0;
        break;
    }
    swap(a[i], a[k]);
    if (i != k)
        det = -det;
    det *= a[i][i];
    for (int j = i + 1; j < n; ++j)
        a[i][j] /= a[i][i];
    for (int j = 0; j < n; ++j)
        if (j != i && abs(a[j][i]) > EPS)
            for (int k = i + 1; k < n; ++k)
                a[j][k] -= a[i][k] * a[j][i];
}
cout << det;
```

6.2 Guass-jordan

```
/*The input to the function gauss is the system
↪  matrix $a$. The last column of this matrix is
↪  vector $b$.
The function returns the number of solutions of the
↪  system $(0, 1,\textrm{or } \infty)$. If at
↪  least one solution exists, then it is returned
↪  in the vector $ans$.*/
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to
↪  be infinity or a big number
int gauss (vector < vector<double> > a,
↪  vector<double> & ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;
    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;
        if (abs (a[sel][col]) < EPS)
            continue;
        for (int i=col; i<=m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }
    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] /
            ↪  a[where[i]][i];
    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0;
    }
    for (int i=0; i<m; ++i)
        if (where[i] == -1)
            return INF;
    return 1;
}
```

7 Matrix

7.1 Integration (Simpsons)

```
#include<bits/stdc++.h>
using namespace std;

#define double long double
const double eps = 1e-6;

int z, d;
double f(double x) {
  double t1 = sqrt(x * x + z * z);
  double t2 = sqrt((x + d) * (x + d) + z * z);
  double w = (t2 - t1) + (d - (t2 - t1)) / 2.0;
  double ans = w * w * 0.5 + t1 * w;
  ans += (d - w) * (d - w) * 0.5 + t2 * (d - w);
  ans /= d;
  return ans;
}
inline double simpson(double fl, double fr, double
↪  fmid, double l, double r){
  return (fl + fr + 4.0 * fmid) * (r - l) / 6.0;
}
double solve(double slr, double fl, double fr,
↪  double fmid, double l, double r){
  double mid = (l + r) / 2;
  double fml = f((l + mid) / 2);
  double fmr = f((mid + r) / 2);
  double slm = simpson(fl, fmid, fml, l, mid);
  double smr = simpson(fmid, fr, fmr, mid, r);
  if (fabs(slr - slm - smr) < eps) return slm + smr;
  return solve(slm, fl, fmid, fml, l, mid) +
  ↪  solve(smr, fmid, fr, fmr, mid, r);
}
// integrate in range [l, r]
double integrate(double l, double r){
```

```
  double mid = (l + r) / 2, fl = f(l), fr = f(r), fmid = f(mid);
  double slr = simpson(fl, fr, fmid, l, r);
  return solve(slr, fl, fr, fmid, l, r);
}
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int t, cs = 0; cin >> t;
  while (t--) {
    int r, l; cin >> z >> r >> l >> d;
    r -= d;
    double ans;
    if (l == r) ans = f(l);
    else ans = integrate(l, r) / (r - l);
    cout << "Case " << ++cs << ": " << fixed << setprecision(10) << ans << '\n';
  }
  return 0;
}
// https://vjudge.net/problem/UVA-12997
```

7.2 Inverse of A Matrix

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9, mod = 1e9 + 7;
int power(long long n, long long k, const int mod) {
  int ans = 1 % mod; n %= mod; if (n < 0) n += mod;
  while (k) {
    if (k & 1) ans = (long long) ans * n % mod;
    n = (long long) n * n % mod;
    k >>= 1;
  }
  return ans;
}
bool Gauss(vector<vector<int>> a, vector<vector<int>> &ans, const int mod) {
  int n = (int)a.size(); vector<int> col(n);
  vector<vector<int>> tmp(n, vector<int>(n));
  for (int i = 0; i < n; i++) tmp[i][i] = 1, col[i] = i;
  for (int i = 0; i < n; i++) {
    int r = i, c = i;
    for (int j = i; j < n; j++) {
      for (int k = i; k < n; k++) {
        if (a[j][k]) {
          r = j; c = k; goto found;
        }
      }
    }
    return false; //not invertible
    found:
    a[i].swap(a[r]); tmp[i].swap(tmp[r]);
    for (int j = 0; j < n; j++) {
      swap(a[j][i], a[j][c]); swap(tmp[j][i], tmp[j][c]);
    }
    swap(col[i], col[c]);
    int v = power(a[i][i], mod - 2, mod);
    for (int j = i + 1; j < n; j++) {
      int f = 1LL * a[j][i] * v % mod;
      a[j][i] = 0;
      for (int k = i + 1; k < n; k++) {
        a[j][k] -= 1LL * f * a[i][k] % mod;
        if (a[j][k] < 0) a[j][k] += mod;
      }
      for (int k = 0; k < n; k++) {
        tmp[j][k] -= 1LL * f * tmp[i][k] % mod;
        if (tmp[j][k] < 0) tmp[j][k] += mod;
      }
    }
    for (int j = i + 1; j < n; j++) a[i][j] = 1LL * a[i][j] * v % mod;
    for (int j = 0; j < n; j++) tmp[i][j] = 1LL * tmp[i][j] * v % mod;
    a[i][i] = 1;
  }
  for (int i = n - 1; i > 0; --i) {
    for (int j = 0; j < i; j++) {
      int v = a[j][i];
      for (int k = 0; k < n; k++) {
        tmp[j][k] -= 1LL * v * tmp[i][k]%mod;
        if (tmp[j][k] < 0) tmp[j][k] += mod;
      }
    }
  }
  ans.resize(n, vector<int>(n));
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
      ans[col[i]][col[j]] = tmp[i][j];
    }
  }
  return true; //invertible
}
int32_t main() {
  int n, mod;
  while (cin >> n >> mod && n && mod) {
    vector<vector<int>> a(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) cin >> a[i][j];
    }
    vector<vector<int>> ans;
    int ok = Gauss(a, ans, mod);
    if (!ok) {
      cout << "singular\n";
      continue;
    }
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) cout << ans[i][j] << ' ';
      cout << '\n';
    }
  }
  return 0;
}
```

7.3 Matrix Exponentiation

```
#include<bits/stdc++.h>
using namespace std;

const int mod = 998244353;

struct Mat {
  int n, m;
  vector<vector<int>> a;
  Mat() { }
  Mat(int _n, int _m) {n = _n; m = _m; a.assign(n, vector<int>(m, 0)); }
  Mat(vector< vector<int> > v) { n = v.size(); m = n ? v[0].size() : 0; a = v; }
  inline void make_unit() {
    assert(n == m);
    for (int i = 0; i < n; i++)  {
      for (int j = 0; j < n; j++) a[i][j] = i == j;
    }
  }
  inline Mat operator + (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
      }
    }
    return ans;
  }
  inline Mat operator - (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] - b.a[i][j] + mod) % mod;
      }
    }
    return ans;
  }
  inline Mat operator * (const Mat &b) {
    assert(m == b.n);
    Mat ans = Mat(n, b.m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < b.m; j++) {
        for(int k = 0; k < m; k++) {
          ans.a[i][j] = (ans.a[i][j] + 1LL * a[i][k] * b.a[k][j] % mod) % mod;
        }
      }
    }
    return ans;
  }
  inline Mat pow(long long k) {
    assert(n == m);
    Mat ans(n, n), t = a; ans.make_unit();
    while (k) {
      if (k & 1) ans = ans * t;
      t = t * t;
      k >>= 1;
    }
    return ans;
  }
  inline Mat& operator += (const Mat& b) { return *this = (*this) + b; }
  inline Mat& operator -= (const Mat& b) { return *this = (*this) - b; }
  inline Mat& operator *= (const Mat& b) { return *this = (*this) * b; }
  inline bool operator == (const Mat& b) { return a == b.a; }
  inline bool operator != (const Mat& b) { return a != b.a; }
```

```
};

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int n, m, k; cin >> n >> m >> k;
  Mat a(n, m);
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
      cin >> a.a[i][j];
    }
  }
  Mat b(m, k);
  for (int i = 0; i < m; i++) {
    for (int j = 0; j < k; j++) {
      cin >> b.a[i][j];
    }
  }
  Mat ans = a * b;
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < k; j++) {
      cout << ans.a[i][j] << ' ';
    }
    cout << '\n';
  }
  return 0;
}
// https://judge.yosupo.jp/problem/matrix_product
```

8 Notes

8.1 Counting

8.1.1 Fibonacci

Let A, B and n be integer numbers.

$$k = A - B \tag{1}$$

$$F_A F_B = F_{k+1} F_A^2 + F_k F_A F_{A-1} \tag{2}$$

$$\sum_{i=0}^{n} F_i^2 = F_{n+1} F_n \tag{3}$$

$$\sum_{i=0}^{n} F_i F_{i+1} = F_{n+1}^2 - (-1)^n \tag{4}$$

$$\sum_{i=0}^{n} F_i F_{i+1} = \sum_{i=0}^{n-1} F_i F_{i+1} \tag{5}$$

$$gcd(F_m, F_n) = F_{gcd(m,n)} \tag{6}$$

$$\sum_{0 \le k \le n} = F_{n+1} \tag{7}$$

$$\tag{8}$$

8.2 Geometry

8.2.1 Triangle

Circumradious: $R = \dfrac{abc}{4A}$, Inradious: $r = \dfrac{A}{s}$

Length of radian: $m_a = \dfrac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector: $s_a = \sqrt{bc[1 - (\frac{a}{b+c})^2]}$

Law of tangents: $\dfrac{a+b}{a-b} = \dfrac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

8.3 Series

$$e^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{3} + \cdots$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \cdots$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4 - \frac{x^6}{6 + \cdots}}$$

8.4 Sum

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{36}$$

9 QOL

9.1 build

```
#include<bits/stdc++.h>
using namespace std;

// {
//  "cmd" : ["g++ -std=c++17 -DLOCAL $file_name -o
    $file_base_name && timeout 4s ./$file_base_name
    <input.txt>output.txt"],"selector" :
    "source.c",
//  "shell":true,
//  "working_dir" : "$file_path"
// }
```

9.2 debug

```
#include<bits/stdc++.h>
using namespace std;

typedef                 long long   ll;
typedef                 long double  lld;
typedef                 vector<ll>   vl;
typedef                 vector<string>  vs;
typedef                 vector<vl>   vvl;
typedef                 pair<ll, ll>  pl;
typedef                 unsigned long long ull;

// Print my stuff
void _print(ll t)           {cerr << t;}
void _print(int t)          {cerr << t;}
void _print(string t)       {cerr << t;}
void _print(char t)         {cerr << t;}
void _print(lld t)          {cerr << t;}
void _print(double t)       {cerr << t;}
void _print(ull t)          {cerr << t;}

// Print my DS
template <class T, class V> void _print(pair <T, V>
    p);
template <class T> void _print(vector <T> v);
template <class T> void _print(set <T> v);
template <class T, class V> void _print(map <T, V>
    v);
template <class T, class V> void
    _print(unordered_map <T, V> v);
template <class T> void _print(multiset <T> v);
template <class T, class V> void _print(pair <T, V>
    p) {cerr << "{"; _print(p.ff); cerr << ",";
    _print(p.ss); cerr << "}";}
template <class T> void _print(vector <T> v) {cerr
    << endl << "[ "; for (T i : v) {_print(i); cerr
    << " ";} cerr << "]";}
template <class T> void _print(set <T> v) {cerr <<
    "[ "; for (T i : v) {_print(i); cerr << " ";}
    cerr << "]";}
template <class T> void _print(multiset <T> v)
    {cerr << "[ "; for (T i : v) {_print(i); cerr
    << " ";} cerr << "]";}
template <class T, class V> void _print(map <T, V>
    v) {cerr << "[ \n"; for (auto i : v)
    {_print(i); cerr << endl;} cerr << "]";}
template <class T, class V> void
    _print(unordered_map <T, V> v) {cerr << "[ ";
    for (auto i : v) {_print(i); cerr << " ";} cerr
    << "]";}

signed main()
{

}
```

9.3 stress

```
#include<bits/stdc++.h>
using namespace std;

signed main()
{
 // set -e
 // g++ code.cpp -o code
 // g++ gen.cpp -o gen
 // g++ brute.cpp -o brute
 // g++ checker.cpp -o checker
 // for((i = 1; ; ++i)); do
 //     ./gen $i > input_file
 //     ./code < input_file > myAnswer
 //     ./brute < input_file > correctAnswer
 //     ./checker > checker_log
 //     echo "Passed test: "  $i
```

```
  // done
}
```

10 String

10.1 KMP

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

// returns the longest proper prefix array of pattern p
// where lps[i]=longest proper prefix which is also suffix of p[0...i]
vector<int> build_lps(string p) {
  int sz = p.size();
  vector<int> lps;
  lps.assign(sz + 1, 0);
  int j = 0;
  lps[0] = 0;
  for(int i = 1; i < sz; i++) {
    while(j >= 0 && p[i] != p[j]) {
      if(j >= 1) j = lps[j - 1];
      else j = -1;
    }
    j++;
    lps[i] = j;
  }
  return lps;
}
vector<int>ans;
// returns matches in vector ans in 0-indexed
void kmp(vector<int> lps, string s, string p) {
  int psz = p.size(), sz = s.size();
  int j = 0;
  for(int i = 0; i < sz; i++) {
    while(j >= 0 && p[j] != s[i])
      if(j >= 1) j = lps[j - 1];
      else j = -1;
    j++;
    if(j == psz) {
      j = lps[j - 1];
      // pattern found in string s at position i-psz+1
      ans.push_back(i - psz + 1);
    }
    // after each loop we have j=longest common suffix of s[0..i] which is also prefix of p
  }
}

int main() {
  int i, j, k, n, m, t;
  cin >> t;
  while(t--) {
    string s, p;
    cin >> s >> p;
    vector<int>lps = build_lps(p);
    kmp(lps, s, p);
    if(ans.empty()) cout << "Not Found\n";
    else {
      cout << ans.size() << endl;
      for(auto x : ans) cout << x << ' ';
      cout << endl;
    }
    ans.clear();
    cout << endl;
  }
  return 0;
}
```

10.2 Manachers

```
#include<bits/stdc++.h>
using namespace std;

int main() {
  int i, j, k, n, m;
  string s;
  cin >> s;
  n = s.size();
  vector<int> d1(n);  // maximum odd length palindrome centered at i
  // here d1[i]=the palindrome has d1[i]-1 right characters from i
  // e.g. for aba, d1[1]=2;
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 1 : min(d1[l + r - i], r - i);
    while (0 <= i - k && i + k < n && s[i - k] == s[i + k]) {
      k++;
    }
    d1[i] = k--;
    if (i + k > r) {
      l = i - k;
      r = i + k;
    }
  }
  vector<int> d2(n);  // maximum even length palindrome centered at i
  // here d2[i]=the palindrome has d2[i]-1 right characters from i
  // e.g. for abba, d2[2]=2;
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
    while (0 <= i - k - 1 && i + k < n && s[i - k - 1] == s[i + k]) {
      k++;
    }
    d2[i] = k--;
    if (i + k > r) {
      l = i - k - 1;
      r = i + k ;
    }
  }
  for(i = 0; i < n; i++) cout << d1[i] << ' ';
  cout << endl;
  for(i = 0; i < n; i++) cout << d2[i] << ' ';
  cout << endl;
  // number of palindromes
  long long ans = 0;
  for(i = 0; i < n; i++) {
    ans += 1LL * d1[i];
    ans += 1LL * d2[i];
  }
  cout << ans << endl;
  return 0;
}
```

10.3 String Hashing 2D

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;

struct Hashing {
  vector<vector<int>> hs;
  vector<int> PWX, PWY;
  int n, m;
  static const int PX = 3731,  PY = 2999, mod = 998244353;
  Hashing() {}
  Hashing(vector<string>& s) {
    n = (int)s.size(), m = (int)s[0].size();
    hs.assign(n + 1, vector<int>(m + 1, 0));
    PWX.assign(n + 1, 1);
    PWY.assign(m + 1, 1);
    for (int i = 0; i < n; i++) PWX[i + 1] = 1LL * PWX[i] * PX % mod;
    for (int i = 0; i < m; i++) PWY[i + 1] = 1LL * PWY[i] * PY % mod;
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < m; j++) {
        hs[i + 1][j + 1] = s[i][j] - 'a' + 1;
      }
    }
    for (int i = 0; i <= n; i++) {
      for (int j = 0; j < m; j++) {
        hs[i][j + 1] = (hs[i][j + 1] + 1LL * hs[i][j] * PY % mod) % mod;
      }
    }
    for (int i = 0; i < n; i++) {
      for (int j = 0; j <= m; j++) {
        hs[i + 1][j] = (hs[i + 1][j] + 1LL * hs[i][j] * PX % mod) % mod;
      }
    }
  }
  int get_hash(int x1, int y1, int x2, int y2) { // 1-indexed
    assert(1 <= x1 && x1 <= x2 && x2 <= n);
    assert(1 <= y1 && y1 <= y2 && y2 <= m);
    x1--;
    y1--;
    int dx = x2 - x1, dy = y2 - y1;
    return (1LL * (hs[x2][y2] - 1LL * hs[x2][y1] * PWY[dy] % mod + mod) % mod -
        1LL * (hs[x1][y2] - 1LL * hs[x1][y1] * PWY[dy] % mod + mod) % mod * PWX[dx] % mod + mod) % mod;
  }
  int get_hash() {
    return get_hash(1, 1, n, m);
  }
};
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  int t;
```

```
  cin >> t;
  while (t--) {
    int n, m;
    cin >> n >> m;
    vector<string> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    Hashing H(a);
    int x, y;
    cin >> x >> y;
    vector<string> b(x);
    for (int i = 0; i < x; i++) cin >> b[i];
    auto z = Hashing(b).get_hash();
    int ans = 0;
    for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= m; j++) {
        if (i + x - 1 <= n && j + y - 1 <= m &&
         ↪  H.get_hash(i, j, i + x - 1, j + y - 1)
         ↪  == z) ans++;
      }
    }
    cout << ans << '\n';
  }
  return 0;
}
// https://vjudge.net/problem/UVA-11019
```

10.4 String Hashing

```
#include<bits/stdc++.h>
using namespace std;

const int N = 1e6 + 9;

int power(long long n, long long k, const int mod) {
  int ans = 1 % mod;
  n %= mod;
  if (n < 0) n += mod;
  while (k) {
    if (k & 1) ans = (long long) ans * n % mod;
    n = (long long) n * n % mod;
    k >>= 1;
  }
  return ans;
}

const int MOD1 = 127657753, MOD2 = 987654319;
const int p1 = 137, p2 = 277;
int ip1, ip2;
pair<int, int> pw[N], ipw[N];
void prec() {
  pw[0] =  {1, 1};
  for (int i = 1; i < N; i++) {
    pw[i].first = 1LL * pw[i - 1].first * p1 % MOD1;
    pw[i].second = 1LL * pw[i - 1].second * p2 %
     ↪  MOD2;
  }
  ip1 = power(p1, MOD1 - 2, MOD1);
  ip2 = power(p2, MOD2 - 2, MOD2);
  ipw[0] =  {1, 1};
  for (int i = 1; i < N; i++) {
    ipw[i].first = 1LL * ipw[i - 1].first * ip1 %
     ↪  MOD1;
    ipw[i].second = 1LL * ipw[i - 1].second * ip2 %
     ↪  MOD2;
  }
}
struct Hashing {
  int n;
  string s; // 0 - indexed
  vector<pair<int, int>> hs; // 1 - indexed
  Hashing() {}
  Hashing(string _s) {
    n = _s.size();
    s = _s;
    hs.emplace_back(0, 0);
    for (int i = 0; i < n; i++) {
      pair<int, int> p;
      p.first = (hs[i].first + 1LL * pw[i].first *
       ↪  s[i] % MOD1) % MOD1;
      p.second = (hs[i].second + 1LL * pw[i].second
       ↪  * s[i] % MOD2) % MOD2;
      hs.push_back(p);
    }
  }
  pair<int, int> get_hash(int l, int r) { // 1 -
   ↪  indexed
    assert(1 <= l && l <= r && r <= n);
    pair<int, int> ans;
    ans.first = (hs[r].first - hs[l - 1].first +
     ↪  MOD1) * 1LL * ipw[l - 1].first % MOD1;
    ans.second = (hs[r].second - hs[l - 1].second +
     ↪  MOD2) * 1LL * ipw[l - 1].second % MOD2;
    return ans;
  }
  pair<int, int> get_hash() {
    return get_hash(1, n);
  }
};
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  prec();
  int n;
  while (cin >> n) {
    string s, p;
    cin >> p >> s;
    Hashing h(s);
    auto hs = Hashing(p).get_hash();
    for(int i = 1; i + n - 1 <= s.size(); i++) {
      if (h.get_hash(i, i + n - 1) == hs) cout << i
       ↪  - 1 << '\n';
    }
    cout << '\n';
  }
  return 0;
}
// https://www.spoj.com/problems/NHAY/
```

10.5 Suffix Array

```
#include<bits/stdc++.h>
using namespace std;

const int N = 3e5 + 9;
const int LG = 18;

void induced_sort(const vector<int> &vec, int
 ↪  val_range, vector<int> &SA, const vector<bool>
 ↪  &sl, const vector<int> &lms_idx) {
  vector<int> l(val_range, 0), r(val_range, 0);
  for (int c : vec) {
    if (c + 1 < val_range) ++l[c + 1];
    ++r[c];
  }
  partial_sum(l.begin(), l.end(), l.begin());
  partial_sum(r.begin(), r.end(), r.begin());
  fill(SA.begin(), SA.end(), -1);
  for (int i = lms_idx.size() - 1; i >= 0; --i)
    SA[--r[vec[lms_idx[i]]]] = lms_idx[i];
  for (int i : SA)
    if (i >= 1 && sl[i - 1]) {
      SA[l[vec[i - 1]]++] = i - 1;
    }
  fill(r.begin(), r.end(), 0);
  for (int c : vec)
    ++r[c];
  partial_sum(r.begin(), r.end(), r.begin());
  for (int k = SA.size() - 1, i = SA[k]; k >= 1;
   ↪  --k, i = SA[k])
    if (i >= 1 && !sl[i - 1]) {
      SA[--r[vec[i - 1]]] = i - 1;
    }
}

vector<int> SA_IS(const vector<int> &vec, int
 ↪  val_range) {
  const int n = vec.size();
  vector<int> SA(n), lms_idx;
  vector<bool> sl(n);
  sl[n - 1] = false;
  for (int i = n - 2; i >= 0; --i) {
    sl[i] = (vec[i] > vec[i + 1] || (vec[i] ==
     ↪  vec[i + 1] && sl[i + 1]));
    if (sl[i] && !sl[i + 1]) lms_idx.push_back(i +
     ↪  1);
  }
  reverse(lms_idx.begin(), lms_idx.end());
  induced_sort(vec, val_range, SA, sl, lms_idx);
  vector<int> new_lms_idx(lms_idx.size()),
   ↪  lms_vec(lms_idx.size());
  for (int i = 0, k = 0; i < n; ++i)
    if (!sl[SA[i]] && SA[i] >= 1 && sl[SA[i] - 1]) {
      new_lms_idx[k++] = SA[i];
    }
  int cur = 0;
  SA[n - 1] = cur;
  for (size_t k = 1; k < new_lms_idx.size(); ++k) {
    int i = new_lms_idx[k - 1], j = new_lms_idx[k];
    if (vec[i] != vec[j]) {
      SA[j] = ++cur;
      continue;
    }
    bool flag = false;
    for (int a = i + 1, b = j + 1;; ++a, ++b) {
      if (vec[a] != vec[b]) {
        flag = true;
        break;
      }
      if ((!sl[a] && sl[a - 1]) || (!sl[b] && sl[b
       ↪  - 1])) {
        flag = !((!sl[a] && sl[a - 1]) && (!sl[b]
         ↪  && sl[b - 1]));
        break;
```

```
      }
    }
    SA[j] = (flag ? ++cur : cur);
  }
  for (size_t i = 0; i < lms_idx.size(); ++i)
    lms_vec[i] = SA[lms_idx[i]];
  if (cur + 1 < (int)lms_idx.size()) {
    auto lms_SA = SA_IS(lms_vec, cur + 1);
    for (size_t i = 0; i < lms_idx.size(); ++i) {
      new_lms_idx[i] = lms_idx[lms_SA[i]];
    }
  }
  induced_sort(vec, val_range, SA, sl, new_lms_idx);
  return SA;
}
vector<int> suffix_array(const string &s, const int
↪ LIM = 128) {
  vector<int> vec(s.size() + 1);
  copy(begin(s), end(s), begin(vec));
  vec.back() = '$';
  auto ret = SA_IS(vec, LIM);
  ret.erase(ret.begin());
  return ret;
}
struct SuffixArray {
  int n;
  string s;
  vector<int> sa, rank, lcp;
  vector<vector<int>> t;
  vector<int> lg;
  SuffixArray() {}
  SuffixArray(string _s) {
    n = _s.size();
    s = _s;
    sa = suffix_array(s);
    rank.resize(n);
    for (int i = 0; i < n; i++) rank[sa[i]] = i;
    costruct_lcp();
    prec();
    build();
  }
  void costruct_lcp() {
    int k = 0;
    lcp.resize(n - 1, 0);
    for (int i = 0; i < n; i++) {
      if (rank[i] == n - 1) {
        k = 0;
        continue;
      }
      int j = sa[rank[i] + 1];
      while (i + k < n && j + k < n && s[i + k] ==
      ↪ s[j + k]) k++;
      lcp[rank[i]] = k;
      if (k) k--;
    }
  }
  void prec() {
    lg.resize(n, 0);
    for (int i = 2; i < n; i++) lg[i] = lg[i / 2] +
    ↪ 1;
  }
  void build() {
    int sz = n - 1;
    t.resize(sz);
    for (int i = 0; i < sz; i++) {
      t[i].resize(LG);
      t[i][0] = lcp[i];
    }
    for (int k = 1; k < LG; ++k) {
      for (int i = 0; i + (1 << k) - 1 < sz; ++i) {
        t[i][k] = min(t[i][k - 1], t[i + (1 << (k -
        ↪ 1))][k - 1]);
      }
    }
  }
  int query(int l, int r) { // minimum of lcp[l],
  ↪ ..., lcp[r]
    int k = lg[r - l + 1];
    return min(t[l][k], t[r - (1 << k) + 1][k]);
  }
  int get_lcp(int i, int j) { // lcp of suffix
  ↪ starting from i and j
    if (i == j) return n - i;
    int l = rank[i], r = rank[j];
    if (l > r) swap(l, r);
    return query(l, r - 1);
  }
  int lower_bound(string &t) {
    int l = 0, r = n - 1, k = t.size(), ans = n;
    while (l <= r) {
      int mid = l + r >> 1;
      if (s.substr(sa[mid], min(n - sa[mid], k)) >=
      ↪ t) ans = mid, r = mid - 1;
      else l = mid + 1;
    }
    return ans;
  }
  int upper_bound(string &t) {
    int l = 0, r = n - 1, k = t.size(), ans = n;
    while (l <= r) {
      int mid = l + r >> 1;
      if (s.substr(sa[mid], min(n - sa[mid], k)) >
      ↪ t) ans = mid, r = mid - 1;
      else l = mid + 1;
    }
    return ans;
  }
  // occurrences of s[p, ..., p + len - 1]
  pair<int, int> find_occurrence(int p, int len) {
    p = rank[p];
    pair<int, int> ans = {p, p};
    int l = 0, r = p - 1;
    while (l <= r) {
      int mid = l + r >> 1;
      if (query(mid, p - 1) >= len) ans.first =
      ↪ mid, r = mid - 1;
      else l = mid + 1;
    }
    l = p + 1, r = n - 1;
    while (l <= r) {
      int mid = l + r >> 1;
      if (query(p, mid - 1) >= len) ans.second =
      ↪ mid, l = mid + 1;
      else r = mid - 1;
    }
    return ans;
  }
};

int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  string s;
  cin >> s;
  SuffixArray sa(s);
  for (int i = 0; i < s.size(); i++) cout <<
  ↪ sa.sa[i] << '\n';
  return 0;
}
```